Cluster: ANLP

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

"Sarathi: A Retrieval-Augmented Assistant for IOE Entrance and Admission Guidance"

Submitted by

Sabin Shrestha (080BCT069)

Sachin K.C. (080BCT070)

A

BE MINOR PROJECT REPORT

SUBMITTED TO THE DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

AUGUST, 2026

Declaration of Authorship

This is to certify that the minor project entitled "Sarathi: A Retrieval-Augmented Assistant for IOE Entrance and Admission Guidance" submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those part of the work with proper citations.

Name of the Student	**Signature**
Sabin Shrestha (080BCT069)	. . . . .
Sachin K.C. (080BCT070)	. . . . .

Date: 27 August 2026

Approval Page

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a minor project entitled "Sarathi: A Retrieval-Augmented Assistant for IOE Entrance and Admission Guidance" submitted by Sabin Shrestha (080BCT069) and Sachin K.C. (080BCT070) in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor, Surendra Shrestha, PhD
Reader,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Internal Examiner,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Head of Department, Prof. Dr. Ram Krishna Maharjan
Department of Electronics & Computer Engineering,
Pulchowk Campus, IOE, TU

Date: 27 August 2026

Copyright

Abstract

Each year several thousand candidates sit the entrance examination conducted by the Institute of Engineering (IOE), Tribhuvan University, for admission to its Bachelor of Engineering and Bachelor of Architecture programmes. The information those candidates need is published, but it is scattered across six independent notice boards, is written largely in Nepali, is dated in the Bikram Sambat calendar, and is distributed as scanned documents that cannot be searched. General-purpose conversational assistants are consulted in this gap and answer confidently and often wrongly, because the year-specific facts an applicant needs – a deadline, a fee, a merit rank, a seat count – are exactly those a language model is least able to recall.

In this work a domain-restricted, retrieval-augmented assistant named Sarathi was designed, implemented and evaluated. Thirteen official notices of the 2083 BS cycle were translated into English and indexed as 216 passages using the multilingual `bge-m3` embedding model, and answers are generated by a locally hosted `qwen2.5:7b` model orchestrated as an explicit state graph performing query rewriting, retrieval, exact lookup, scope adjudication and grounded generation. The organising principle of the work is that every quantity a student may act upon is computed rather than generated: a 7,179-row pass list is answered by exact lookup, fee totals are summed from published line items and checked against the printed totals, Bikram Sambat dates are converted before the prompt is assembled, and last year's Pulchowk programme cut-offs are reconstructed by simulating the published priority applications under the serial-dictatorship rule IOE actually applies.

Evaluation targeted the failure modes the system was built to remove. A three-signal scope guard raised classification from 26/39 to 36/39 on a fixed probe set and refused 10/10 off-topic questions in live verification while answering 10/10 genuine ones. Deterministic removal of language-switching requests produced English answers on 24 of 24 turns, where four prompt-only attempts had failed. Enlarging the context window from the served default of 4,096 to 16,384 tokens proved necessary, a representative grounded prompt assembling 5,318 tokens. All twenty published fee totals reproduced exactly, and the cut-off simulation resolved sixteen programme–category pairs over 1,647 deduplicated applicants. The application runs entirely on commodity hardware with no external model interface.

Keywords: retrieval-augmented generation, large language model, IOE entrance examination, hallucination mitigation, Bikram Sambat, serial dictatorship

Acknowledgement

The authors wish to express their sincere gratitude to the Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering, Tribhuvan University, for providing the opportunity and the academic framework within which this minor project was undertaken.

Profound appreciation is owed to the project supervisor, Surendra Shrestha, PhD, Reader, Department of Electronics & Computer Engineering, whose guidance shaped the direction of this work and whose insistence on measurable claims rather than plausible ones is reflected throughout the evaluation reported here.

The authors are grateful to Prof. Dr. Ram Krishna Maharjan, Head of the Department of Electronics & Computer Engineering, and to the members of the minor project evaluation committee, for their supervision of the project programme and for the review of the proposal from which this work proceeded.

Acknowledgement is also due to the IOE Entrance Examination Board and to the administrations of Pulchowk Campus, Pashchimanchal Campus and Purwanchal Campus, whose published notices constitute the entire factual basis of this system. The open-source communities behind LangGraph, Chroma, Ollama, FastAPI and Next.js are thanked for the software on which the implementation rests.

Finally, the authors thank their classmates, who tested the assistant with questions that no design document had anticipated, and their families, for their patience over the course of the work.

Sabin Shrestha (080BCT069)
Sachin K.C. (080BCT070)

Table of Contents

List of Figures

List of Tables

Abbreviations

AD	Anno Domini (Gregorian calendar)
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
BArch	Bachelor of Architecture
BE	Bachelor of Engineering
BS	Bikram Sambat (Nepali calendar)
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DPR	Dense Passage Retrieval
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IOE	Institute of Engineering
JSON	JavaScript Object Notation
LLM	Large Language Model
NPR	Nepalese Rupee
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
SQL	Structured Query Language
SSE	Server-Sent Events
SSR	Server-Side Rendering
TU	Tribhuvan University
UI	User Interface
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
YAML	YAML Ain't Markup Language

Chapter 1. Introduction

1.1. Background

The Institute of Engineering (IOE), a technical institute of Tribhuvan University, conducts a single common entrance examination that governs admission to the Bachelor of Engineering (BE) and Bachelor of Architecture (BArch) programmes of its constituent and affiliated campuses. Pulchowk Campus, Thapathali Campus, Purwanchal Campus, Pashchimanchal Campus and Chitwan Engineering Campus admit through this examination, as do a substantial number of affiliated colleges. The examination is therefore the single largest gate into undergraduate engineering education in Nepal, and the administrative process that surrounds it – application, correction, fee payment, examination, result publication, quota verification, priority declaration and campus admission – is correspondingly elaborate.

That process is documented, and documented well. For the 2083 BS cycle the IOE Entrance Examination Board published a thirty-two page information booklet, a detailed examination notice, a revised examination notice, a syllabus, a result-publication notice with a 122-page pass list, quota-document notices for domestic and foreign applicants, correction instructions, four payment walkthroughs, and a campus-level admission notice for Pulchowk carrying the complete fee schedule. Each document is authoritative. Collectively, however, they are difficult for their intended reader to use, for four reasons that compound one another.

First, the documents are dispersed. They are not published in one place but across the notice boards of the Entrance Examination Board, the Institute of Engineering, Tribhuvan University, and each campus separately – six independent web sites in the case examined by this project, each with its own layout, its own ordering and its own archival policy.

Second, they are written in Nepali. The booklet, the entrance notices and the quota notices are Nepali throughout; only the syllabus and the payment walkthroughs are in English. An applicant who reads Nepali fluently is not disadvantaged, but the machine-readability of the corpus is, and any system that wishes to reason over these documents must first confront the language.

Third, every date is stated in Bikram Sambat. A notice that gives a deadline of 2083/04/26 is unambiguous to a Nepali reader, but the arithmetic that turns it into “eleven days from now” is not arithmetic that any general-purpose computational system performs reliably, and the Bikram Sambat calendar has irregular month lengths that vary from year to year and cannot be derived by formula.

Fourth, and most consequentially, the documents are published as scanned Portable Document Format (PDF) files. They are images of paper. They cannot be searched, copied, or indexed, and the 122-page result list – 7,179 successful candidates and their merit ranks – is a scanned table.

Into this gap the applicant brings a general-purpose conversational assistant. Large language models have made such assistants ubiquitous, and a candidate who asks one when the IOE entrance examination is held, or what a Regular seat at Pulchowk costs for the full degree, will receive a fluent and confident answer. The answer is frequently wrong. This is not a defect peculiar to any one model. It is the well-documented tendency of generative language models to produce fluent text unsupported by any source, described in the literature as hallucination [1], and it is at its most severe precisely where this domain lives: in facts that are specific to one administrative year, that change annually, and that are underrepresented or absent in any pretraining corpus.

Retrieval-augmented generation (RAG), introduced by Lewis *et al.* [2], addresses this by separating the two things a question-answering system must do. Knowledge is retrieved from an external, inspectable corpus at query time; language is generated by the model conditioned on what was retrieved. The model is relieved of the obligation to remember, and the corpus can be corrected, extended and audited without retraining anything. It is on this architecture that the present work is built.

1.2. Problem Statement

An applicant to the IOE entrance examination requires accurate answers to a small number of consequential questions – when to apply, what to submit, how much to pay, whether they passed, and in what order to declare their campus priorities. The information exists in published form but is not accessible in the form the question is asked. Specifically:

1. The authoritative documents are dispersed across six notice boards, are predominantly in Nepali, and are distributed as non-searchable scanned PDFs, so no direct search of them is possible.
2. Deadlines are expressed in Bikram Sambat, and the conversion to the Gregorian calendar and to a relative offset from the present day is not performed reliably by generative models.
3. General-purpose assistants, consulted in the absence of anything better, answer year-specific questions on fees, dates, seat counts and results from parametric memory, and thereby produce fluent misinformation on exactly the matters where an error costs an applicant a place.

4. Several classes of question in this domain are not retrieval problems at all but exact-computation problems. Whether a form number appears on a published pass list, what a Regular candidate pays over eight semesters, and how far down the merit list a programme admitted last year are questions with single correct answers that must be looked up or computed, never approximated by nearest-neighbour search or by arithmetic performed inside a language model.
5. An assistant that answers questions outside its competence damages the conversation that follows. An off-topic answer, once written into a transcript, is retrieved against and reasoned over on every subsequent turn, so a single lapse of scope degrades the whole session.

The problem addressed by this project is therefore the construction of a domain-restricted assistant that answers IOE admission questions from the official notices alone, that computes rather than generates every figure a student may act upon, that states which document each answer came from, and that declines plainly whatever the notices do not cover.

1.3. Objectives

1.3.1. General Objectives

To design, implement and evaluate a retrieval-augmented conversational assistant that provides accurate, source-attributed and calendar-aware answers to questions concerning the IOE entrance examination and the admission process that follows it, operating entirely on locally hosted models.

1.3.2. Specific Objectives

1. To assemble and translate a corpus of official IOE admission notices and index it in a persistent vector store, and to implement retrieval and generation over it as an explicit state graph whose stages – query rewriting, retrieval, exact lookup, scope adjudication and answer generation – are separately addressable.
2. To answer exact-value questions – pass-list membership and merit rank, admission fee totals, historical programme cut-offs, and Bikram Sambat date arithmetic – by direct lookup and deterministic computation rather than by generation, supplying the results to the model as authoritative context.
3. To attribute each answer to the documents it visibly drew upon, and to restrict the assistant to its domain by an explicit scope guard that refuses anything the notices do not cover.

4. To deliver the system through a responsive web interface with streaming answers, an aggregated notice feed and an extracted deadline list, and to evaluate it against the specific failure modes it was built to remove.

1.4. Rationale

The rationale for this project rests on three observations.

The first concerns the cost of error in this particular domain. An applicant who receives a wrong answer about a Python function loses a few minutes; an applicant who receives a wrong answer about a document-submission deadline may lose an academic year. The population asking these questions is large, is under time pressure, and is often unfamiliar with administrative process. A system operating here must be built so that its most consequential outputs cannot be wrong, rather than merely unlikely to be wrong, and that requirement is what motivates the computed-rather-than-generated principle that organises the entire implementation.

The second concerns the suitability of retrieval augmentation to this corpus. The document set is small, closed, authoritative and revised annually. These are precisely the conditions under which retrieval augmentation outperforms fine-tuning: the knowledge is externalised, so next year's cycle is accommodated by replacing thirteen documents rather than by retraining a model, and every answer remains traceable to a notice the student can open and read.

The third concerns local execution. Both the generation and the embedding models run on the user's own machine through Ollama, and no external model interface is contacted. This removes recurring cost, removes the dependency on network availability that a Nepali deployment cannot safely assume, and – since the questions asked of this system name real candidates and their examination results – keeps personally identifying queries from leaving the machine at all. It also constrains the design honestly: a seven-billion-parameter model is a weaker instrument than a frontier model, and much of the engineering reported in Chapters 4 and 5 exists because that weakness had to be designed around rather than assumed away.

1.5. Scope

The system covers the BE and BArch entrance examination and admission process of the Institute of Engineering, Tribhuvan University, for the 2083 BS (2026 AD) admission cycle. Within that boundary it addresses eligibility, the application procedure, correction of submitted applications, examination structure and syllabus, fee payment through the four published electronic channels, quota documentation for domestic and foreign

applicants, result publication, campus and programme structure, admission deadlines, and the declaration of campus priorities.

Four capabilities are provided beyond conversational retrieval. Exact lookup over the published pass list resolves a query by form number, merit rank, candidate name or district. Deterministic fee computation produces every total a fee question can require for each of the four fee categories at Pulchowk Campus. A historical priority analysis reconstructs, for Pulchowk Campus only, the last-admitted merit rank of each programme in the 2083 cycle. An aggregated notice feed collects listings from six official notice boards and presents them with publication dates in both calendars.

The following are outside the scope of this work. Institutions other than IOE are not covered. Postgraduate admission is acknowledged by the assistant but is not supported by the indexed corpus. Priority analysis is confined to Pulchowk Campus, that being the only campus whose priority applications are published in a form that permits reconstruction. The system does not transact: it does not submit applications, make payments, or alter any record held by IOE. It is an informational assistant, and the official notices remain the sole authority.

1.6. Limitations

The limitations of the delivered system are stated plainly here and revisited against measurement in Section 5.2.

1. **Corpus currency.** The indexed corpus is a snapshot of the 2083 BS cycle. Documents published after indexing are visible to the assistant only as notice-feed listings – title, publisher and date – because the notices themselves are scanned PDFs behind a link and are not fetched. The assistant can therefore report that a newer notice exists but cannot report what it says.
2. **Translation dependency.** Retrieval operates over English translations prepared before indexing. A translation error propagates to every answer drawn from that passage, and the original Nepali PDF, not the translation, remains authoritative.
3. **Model capacity.** Generation uses a seven-billion-parameter model chosen to fit commodity hardware. Its instruction-following is appreciably weaker than that of larger models, and several design decisions documented in Section 4.7 exist solely to compensate.
4. **Coverage of exact computation.** Fee computation covers Pulchowk Campus, whose schedule alone is published in full. Priority analysis likewise covers Pulchowk only, and its figures describe the 2083 cycle; cut-offs move each year with the applicant

pool and the seat allocation, so they are guidance for ordering preferences and never a prediction.

5. **Absence of authentication.** Conversations are scoped to a random browser-held identifier rather than to an account. This separates one user's history from another's but is explicitly not a security boundary.
6. **Latency.** Local inference on a consumer graphics card yields answers in seconds rather than milliseconds, and a grounded answer requires a query-rewrite call, an embedding call and a generation call in sequence.
7. **Evaluation scale.** The system was evaluated against curated probe sets of tens of questions, sized to the failure modes under investigation. No large-scale user study was conducted, and no figure in Chapter 5 should be read as a population-level accuracy estimate.

1.7. Outline of the Report

The remainder of this report is organised as follows.

Chapter 2 reviews the theory on which the system rests – language models, dense retrieval, vector similarity search, retrieval-augmented generation, and the matching-theoretic result that underlies the priority analysis – and surveys related systems, closing with an analysis of the gap this work occupies.

Chapter 3 presents the methodology: the project approach, the system architecture, the module decomposition, the technology stack, the development methodology followed, the handling of data, the testing and validation strategy, and the deployment configuration.

Chapter 4 details the implementation. It states the functional and non-functional requirements, describes the development environment, and works through the implementation of each subsystem – ingestion and indexing, retrieval and reranking, scope adjudication, exact lookup, fee computation, priority simulation, calendar grounding, citation filtering, streaming delivery and the user interface – before recording the principal technical challenges encountered and their resolutions.

Chapter 5 reports the results obtained, quantitatively where quantities were measured, and discusses what they establish and what they do not, including a comparison against the baseline configurations the final system replaced.

Chapter 6 draws conclusions and sets out recommendations for future work. The report closes with references in IEEE style and with appendices containing the application programming interface specification, the programme-code mapping used by the priority simulation, and representative transcripts.

Chapter 2. Literature Review

2.1. Review of Related Theory

2.1.1. Language Models and the Transformer

Modern generative language modelling rests on the transformer architecture introduced by Vaswani *et al.* [3], which replaced recurrence with scaled dot-product self-attention and thereby made it practical to train very large sequence models. A decoder-only transformer defines an autoregressive distribution over a token sequence $w = (w_1, w_2, \ldots, w_n)$ by the chain rule,

$$p_\theta(w) \;=\; \prod_{t=1}^{n} p_\theta(w_t \mid w_1, \ldots, w_{t-1}), \tag{1}$$

where θ denotes the model parameters, w_t the token generated at position t, and n the sequence length. Models trained at sufficient scale under this objective acquire in-context learning: the ability to perform a task specified only by instructions and examples placed in the prompt, without any gradient update. The entire practice of prompt engineering, on which this project depends heavily, follows from that property.

Two consequences of the formulation matter to the present work. The first is that p_θ carries no notion of truth. The objective rewards likelihood, and a fluent falsehood is frequently more likely under θ than a hedged truth – which is the mechanism behind hallucination as surveyed by Ji *et al.* [1]. The second is the finite context window. The conditioning sequence is bounded, and content that exceeds the bound is discarded by the serving layer, generally without error. Section 4.7 records a failure in this project caused entirely by that behaviour.

2.1.2. Text Embeddings and Dense Retrieval

An embedding model is a function $E : \mathscr{T} \to \mathbb{R}^d$ mapping a text passage to a fixed-length vector whose geometry encodes semantic similarity. Sentence-BERT [4] established the now-standard approach of fine-tuning a bidirectional transformer encoder with a contrastive objective, so that passages of similar meaning are placed near one another under a simple metric.

Similarity between a query embedding $\mathbf{q} = E(q)$ and a passage embedding $\mathbf{d} = E(d)$ is measured by the cosine of the angle between them:

$$\mathrm{sim}(\mathbf{q},\mathbf{d}) \;=\; \cos\theta \;=\; \frac{\mathbf{q}\cdot\mathbf{d}}{\|\mathbf{q}\|_2\,\|\mathbf{d}\|_2} \;=\; \frac{\sum_{i=1}^{d} q_i d_i}{\sqrt{\sum_{i=1}^{d} q_i^2}\,\sqrt{\sum_{i=1}^{d} d_i^2}}, \tag{2}$$

where q_i and d_i are the i-th components of the query and passage vectors respectively, d is the embedding dimensionality, and $\mathrm{sim} \in [-1,1]$, with unity denoting identical direction. Cosine similarity is preferred over Euclidean distance because it is invariant to vector magnitude, and passage length correlates with magnitude in ways that carry no semantic content.

Vector stores expose the equivalent cosine *distance*, $d_{\cos} = 1 - \mathrm{sim}$, and report a bounded relevance score

$$r(q,d) \;=\; 1 - d_{\cos}(\mathbf{q},\mathbf{d}) \;=\; \mathrm{sim}(\mathbf{q},\mathbf{d}), \tag{3}$$

where $r(q,d)$ is the relevance of passage d to query q. All numerical thresholds quoted in this report are expressed on this scale.

Karpukhin *et al.* [5] showed with Dense Passage Retrieval that a dual-encoder trained in this manner outperforms strong lexical baselines such as BM25 on open-domain question answering, particularly where the query and the passage share meaning without sharing vocabulary. That property is decisive for the present corpus, whose source documents are Nepali and whose queries are typically English.

The embedding model adopted here is BGE-M3 [6], which is explicitly multilingual, supports long inputs, and unifies dense, sparse and multi-vector retrieval in a single model. Its multilingual alignment is what permits an English query to match text derived from Nepali notices.

2.1.3. Approximate Nearest Neighbour Search

Exhaustive comparison of a query against every indexed passage costs $O(Nd)$ for N passages of dimension d. Practical vector stores instead employ approximate nearest-neighbour search. The Hierarchical Navigable Small World (HNSW) algorithm of Malkov and Yashunin [7] builds a layered proximity graph in which greedy descent from a sparse upper layer to a dense lower layer locates near neighbours in approximately logarithmic time, trading a small and tunable loss of recall for a large gain in throughput. HNSW with a cosine metric is the index configuration used by this project.

2.1.4. Retrieval-Augmented Generation

Retrieval-augmented generation [2] conditions the generator on passages fetched from an external corpus at inference time. Where q is the query, $\mathscr{R}(q)$ the retrieved passage set and y the generated answer, the generation distribution becomes

$$p_\theta(y \mid q, \mathscr{R}(q)) \;=\; \prod_{t=1}^{|y|} p_\theta(y_t \mid y_{<t}, q, \mathscr{R}(q)), \tag{4}$$

in which y_t is the t-th generated token and $y_{<t}$ the tokens preceding it. The knowledge the answer depends upon resides in $\mathscr{R}(q)$ rather than in θ, and is therefore inspectable, correctable and citable.

Shuster *et al.* [8] demonstrated empirically that retrieval augmentation substantially reduces hallucination in knowledge-grounded dialogue. Gao *et al.* [9] survey the design space that has since developed, distinguishing naive, advanced and modular RAG and identifying chunking policy, query transformation and post-retrieval reranking as the components that most affect answer quality. The pipeline described in Chapter 3 is modular in that taxonomy.

2.1.5. Chunking, Query Transformation and Reranking

A document must be divided into passages before it can be embedded, because an embedding of a whole document dilutes every specific claim it contains. Header-aware splitting preserves the document's own logical divisions; a recursive character splitter then bounds the size of any section that remains too large, with a fixed overlap so that a table or list divided at a boundary remains readable on both sides. For a document of L characters split at maximum size s with overlap o, the number of chunks is bounded by

$$n_{\text{chunks}} \;=\; \left\lceil \frac{L-o}{s-o} \right\rceil, \qquad 0 \le o < s, \tag{5}$$

where L is document length in characters, s the maximum chunk size and o the overlap.

Conversational retrieval requires a further step. A follow-up such as "what about the fees?" retrieves nothing useful in isolation because its subject is carried by the preceding turns. Query rewriting condenses the message against the conversation into a standalone query before embedding, a transformation identified by Gao *et al.* [9] as among the highest-value pre-retrieval operations.

Post-retrieval reranking reorders candidates by criteria the embedding does not capture. Reranking need not be neural: a domain rule that demotes passages addressed to a minority audience is a reranking function, and, as Section 4.4 shows, an effective one.

2.1.6. Agentic Orchestration as a State Graph

Once a pipeline acquires conditional stages – skip retrieval for a greeting, refuse an out-of-domain question, consult an exact table when the question contains a key – a linear chain ceases to describe it. Asai *et al.* [10] showed in Self-RAG that a system which decides when to retrieve and critiques what it retrieved outperforms one that always retrieves.

The present system adopts the same insight with the control flow made explicit rather than delegated to the model. The pipeline is a directed graph $G = (V, E)$ whose vertices are functions over a shared typed state and whose conditional edges are ordinary predicates, evaluated in code. This is a deliberate design position: a seven-billion-parameter model is not a reliable router, and every routing decision it was asked to make in early versions of this project it eventually got wrong.

2.1.7. Serial Dictatorship and the Reconstruction of Cut-offs

IOE admits candidates to campus programmes by a rule that is well understood in matching theory. Candidates are processed in merit order, and each is assigned to the highest-ranked programme on their own declared priority list that still has a seat available. This is the *serial dictatorship* mechanism, a special case of the priority-based assignment analysed by Abdulkadiroğlu and Sönmez [11] within the tradition established by Gale and Shapley [12].

Two properties of serial dictatorship are used directly by this project. It is *strategy-proof*: no candidate can improve their own assignment by declaring a priority order other than their true preference, which is precisely the advice the assistant is required to give. And it is *deterministic*: given the merit order, the declared priority lists and the seat counts, the allocation is uniquely determined and therefore reproducible. Since IOE publishes all three inputs, last year's cut-off for every programme – the merit rank of the last candidate admitted – can be reconstructed exactly, although no such table is published anywhere. Section 4.4 gives the algorithm and Section 5.1 the reconstructed cut-offs.

2.1.8. The Bikram Sambat Calendar

Bikram Sambat is the official solar calendar of Nepal, running approximately 56.7 years ahead of the Gregorian calendar. Its months are of unequal and *year-varying* length, between 29 and 32 days, determined by solar transit rather than by rule. There is consequently no closed-form conversion between Bikram Sambat and Gregorian dates; correct conversion requires a lookup table of month lengths per year. This is the reason calendar handling in this project is delegated to a tabulated calendar library rather than computed, and the reason no generative model can be trusted with it.

2.2. Review of Related Works

2.2.1. Conversational Assistants in Higher Education

Chatbots have been applied to student services for over a decade. Wollny *et al.* [13] systematically reviewed the field and found that the largest single application area is administrative and advisory support – admissions, enrolment and scheduling – rather than tutoring, and that the great majority of deployed systems were rule-based or intent-classified rather than generative. Such systems answer reliably within their scripted intents and fail entirely outside them.

The relevance to this work is the trade-off those reviews describe. An intent-classified assistant cannot hallucinate, because it can only emit authored responses, but it also cannot answer anything its authors did not anticipate, and the administrative surface of IOE admission is far too large to enumerate. A generative assistant answers anything and is therefore untrustworthy. The architecture pursued here attempts to obtain the coverage of the second and the reliability of the first, by generating the language while computing the facts.

2.2.2. Retrieval-Augmented Question Answering over Institutional Corpora

A considerable body of applied work now indexes institutional documentation for retrieval-augmented question answering: policy manuals, statutory instruments, clinical guidelines and course catalogues. The consistent finding, summarised by Gao *et al.* [9], is that domain performance is determined less by the choice of generator than by corpus preparation, chunking policy and the discipline of the grounding prompt. This project's experience supports that conclusion: every substantial improvement recorded in Chapter 5 came from the pipeline surrounding the model rather than from the model.

2.2.3. Hallucination Mitigation and Attribution

Beyond retrieval itself, two families of mitigation are relevant. Self-RAG [10] trains a model to emit reflection tokens that critique whether its output is supported by the retrieved evidence. SelfCheckGPT [14] detects hallucination without any external resource by sampling several generations and measuring their mutual consistency. Both are effective and both are costly, requiring either a specially trained model or several generations per answer.

Neither is affordable at seven billion parameters on a single consumer graphics card serving a streaming interface. The approach adopted here is instead a lexical grounding test applied after generation, described in Section 4.4: a retrieved document is cited only if the finished answer shares a sufficient quantity of distinctive vocabulary with it. This

is weaker than either published method and was chosen for its cost, but it addresses the specific failure that motivated it – a refusal printed above a list of sources it never consulted.

2.2.4. Natural Language Processing for Nepali

Nepali remains a low-resource language in natural language processing. Timilsina *et al.* [15] report NepBERTa, a monolingual transformer trained on the largest Nepali corpus assembled at that time, and note the scarcity of both corpora and downstream benchmarks. For a system that must retrieve over Nepali source material, two strategies are available: employ a Nepali-capable encoder directly, or translate the corpus into English at ingestion and retrieve monolingually. The second was adopted here, for reasons set out in Section 3.6 – chiefly that the generator reads English far better than it translates while reasoning, and that translation performed once at ingestion is auditable in a way that translation performed at inference is not.

2.2.5. Systems Closest to This Work

Several publicly available services list IOE notices, and several private tutoring platforms publish entrance preparation material and rank predictors. They fall into two groups. The first are notice aggregators, which reproduce listings without answering questions about their content. The second are rank or cut-off predictors, which project admission thresholds from historical figures; these are typically neither sourced nor reproducible, and they present projected figures as advice. No system was found that answers free-form questions from the official notices, cites the notice each answer came from, or reconstructs programme cut-offs from the published priority applications rather than estimating them.

2.3. Gap Analysis

Table 2 summarises the gap this work occupies with respect to the approaches reviewed above.

Table 2. Gap analysis against existing approaches

Capability	**Rule-based service chatbot**	**General-purpose LLM**	**This work**
Open-ended questions	No; scripted intents only	Yes	Yes
Factual grounding	Authored answers	Parametric memory; unreliable	Retrieved from official notices
Source attribution	Not applicable	Absent or fabricated	Per-answer, filtered by use
Exact values (results, fees, cut-offs)	Only if scripted	Generated; frequently wrong	Looked up or computed
BS/AD calendar handling	Hard-coded	Unreliable	Computed before generation
Behaviour outside domain	Falls through	Answers anyway	Explicit refusal
Currency of information	Manual edit	Fixed at training	Re-index; live notice feed
Data residency	Local	External service	Fully local inference

Three gaps are addressed specifically by this project. First, no existing system answers IOE admission questions from the official notices with per-answer attribution to the notice used. Second, no existing system treats the exact-value questions of this domain as computation rather than generation, although those are the questions on which an applicant most relies. Third, no published source reconstructs Pulchowk programme cut-offs from the priority applications, despite the underlying mechanism being deterministic and all its inputs being published; the reconstruction reported in Section 5.1 appears to be the first of its kind for this admission cycle.

Chapter 3. Methodology

This project delivers a working software product rather than an experimental study, and the methodology is presented accordingly, following the product-based branch of the prescribed outline. Where research-oriented concerns genuinely apply – the evaluation of retrieval quality and the reranking scheme – they are treated within the sections below rather than duplicated as a separate branch.

3.1. Project Approach

The approach adopted proceeds from a single organising principle, which is stated here because every subsequent decision follows from it:

> *The language model writes the sentences. It does not determine the facts. Any quantity a student may act upon – a date, an amount, a rank, a cut-off – is retrieved from a document or computed in code, and is placed in the prompt already settled.*

This principle divides the domain into two classes of question, handled by different machinery. *Expository* questions – what documents a quota applicant must submit, how the entrance examination is structured, how a fee is paid through eSewa – are answered by retrieval over the indexed corpus, because their answers are passages of text. *Exact-value* questions – whether a form number appears on the pass list, what a Regular candidate pays across eight semesters, how far down the merit list Computer Engineering admitted last year – are answered by direct lookup or deterministic computation, because their answers are single correct values that nearest-neighbour search cannot be trusted to return and that arithmetic performed inside a language model cannot be trusted to produce.

Four further commitments follow from the principle:

1. **Translate at ingestion, not at inference.** The Nepali corpus is translated into structured English Markdown once, with provenance recorded in document front matter, so that the translation is auditable and retrieval is monolingual.
2. **Make control flow explicit.** Routing decisions are predicates in code, evaluated over a typed state object, not instructions the model is asked to obey.
3. **Attribute after generation, not before.** A document is cited only if the finished answer visibly used it.

4. **Refuse rather than approximate.** Where the corpus does not cover a question, or the question lies outside the domain, the system says so in fixed wording of its own.

3.2. System Architecture

The system is a three-tier application. A Next.js client renders the interface and consumes a streaming response delivered as Server-Sent Events; a FastAPI service exposes the conversational, notice and administrative endpoints; and a LangGraph state machine within that service orchestrates retrieval, exact lookup, scope adjudication and generation against a Chroma vector store and a locally hosted Ollama runtime. Persistence is provided by an on-disk vector index, a SQLite database holding conversation checkpoints and the thread index, and a JSON cache of scraped notices. Figure 1 shows the arrangement.

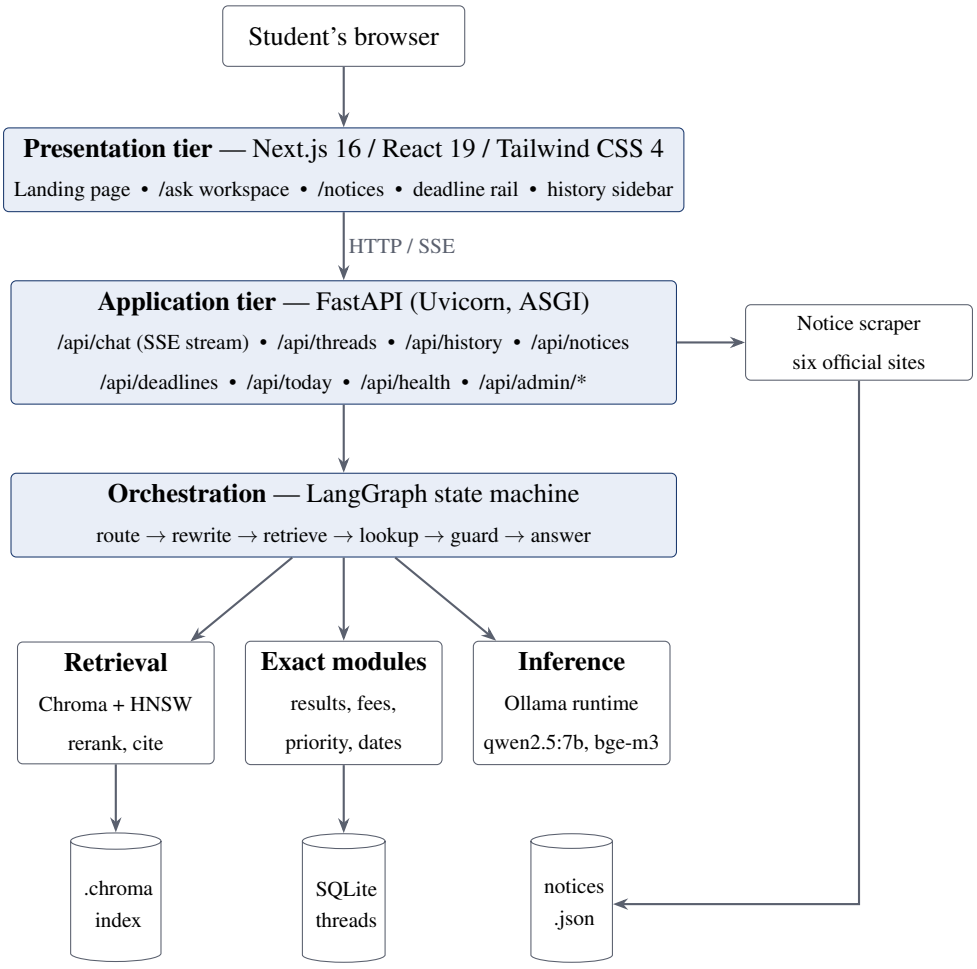

Figure 1. Overall system architecture of Sarathi

The tiers communicate only over the public HTTP interface. The client is a statically built bundle that calls the application programming interface from the browser rather than from the server, which is why the interface base address is compiled into the bundle

at build time rather than resolved at run time. This detail governs the deployment configuration described in Section 3.8.

3.3. System Design and Modules

The backend is decomposed into eleven modules, each with one responsibility. Table 3 lists them with their size, measured in source lines.

Table 3. Backend module decomposition and responsibilities

Module	Lines	Responsibility
`graph.py`	748	State-graph definition; system prompt; query rewriting; language normalisation; small-talk routing; scope guard; answer assembly and citation selection
`results.py`	490	Exact lookup over the 7,179-row pass list by form number, merit rank, candidate name or district
`api.py`	422	FastAPI application; SSE streaming; thread, notice, deadline and administrative endpoints
`rag.py`	403	Document loading and chunking; Chroma index construction; retrieval, reranking, citation payload and grounding filter
`fees.py`	377	Deterministic arithmetic over the published Pulchowk fee tables, with self-verification against printed totals
`notices.py`	358	Scrapers for six official notice boards; cache; digest for prompt injection
`priority.py`	338	Serial-dictatorship simulation of the 2083 Pulchowk priority applications; cut-off derivation and guidance
`threads.py`	261	Conversation index behind the history sidebar; model-written thread titles
`deadlines.py`	142	Extraction of dated obligations from the indexed documents
`dates.py`	124	Current date in both calendars; Bikram Sambat conversion and day-offset computation
`main.py`	32	Terminal chat loop for development use
Total	**3,695**	

3.3.1. The Orchestration Graph

The conversational pipeline is a directed graph over a typed state object. Figure 2 shows its nodes and edges, including the two conditional branches that make it a graph rather than a chain.

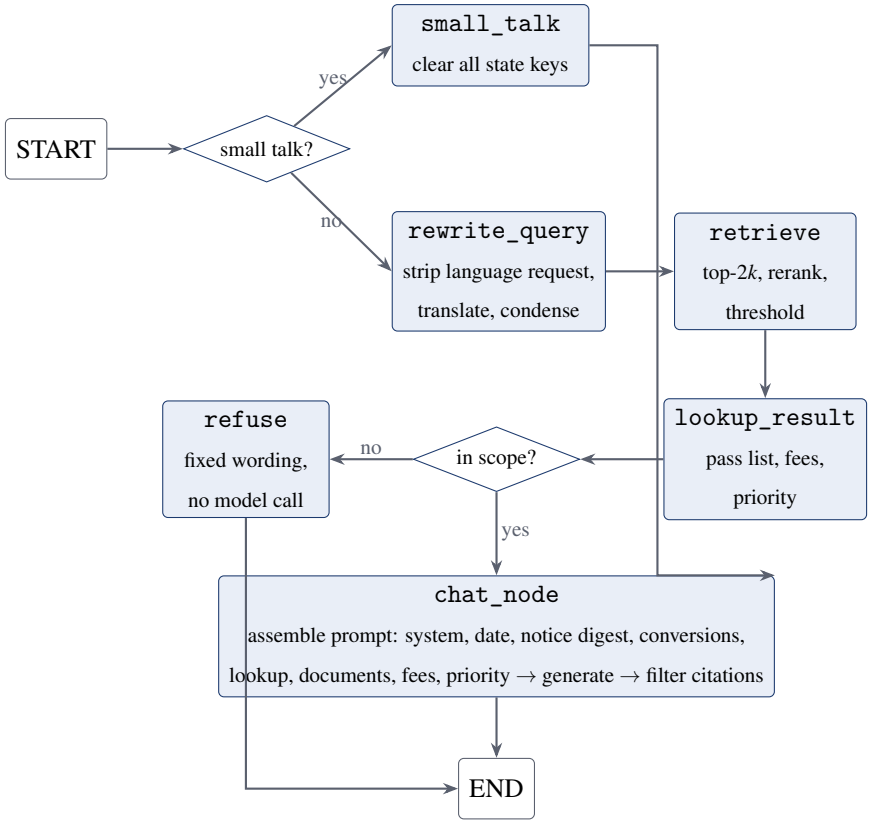

Figure 2. Node and edge structure of the orchestration graph

The state carried between nodes holds the conversation transcript, the normalised question, the rewritten search query, the retrieved context, the exact-lookup block, the computed fee block, the priority-analysis block, the candidate citations and any refusal issued by the guard. Two properties of this design are relied upon throughout the implementation. Because the state persists across turns, every node that may leave a key unwritten must instead write it empty – otherwise a turn that retrieves nothing inherits and cites the previous turn's documents. And because the graph is compiled with a checkpointer backed by SQLite, a conversation survives a restart of the service.

3.3.2. Frontend Modules

The client comprises three routes and fifteen components. The landing route presents the system's three substantive claims and a notice rail. The `/ask` route is a three-column workspace: conversation history on the left, transcript and composer in the centre, notice rail on the right, with each column scrolling independently above the extra-large breakpoint and collapsing to a single scrolling page below it. The `/notices` route lists the aggregated feed. Supporting components render Markdown answers, source citations, the extracted deadline list, the dual-calendar dateline and the theme toggle.

3.4. Technology Stack

Table 4 states the technology stack with the versions against which the system was developed and tested.

Table 4. Technology stack and versions

Layer	Technology	Version	Role
Language (backend)	Python	3.12.3	Backend implementation
Language (frontend)	TypeScript	5.x	Client implementation
Generation model	`qwen2.5:7b`	7B params	Answer, rewrite, title, scope classification
Embedding model	`bge-m3`	latest	Multilingual passage and query embedding
Model runtime	Ollama	0.32.6	Local inference server
Orchestration	LangGraph	1.2.11	State-graph pipeline
LLM bindings	LangChain (Ollama, Chroma, text-splitters)	1.1.x	Model, store and splitter interfaces
Vector store	Chroma	1.1.0	Persistent HNSW index, cosine metric
Checkpointing	`langgraph-checkpoint-sqlite`	3.1.1	Conversation persistence
Web framework	FastAPI	0.141.1	REST and SSE endpoints
ASGI server	Uvicorn	0.52.4	Serving
Calendar	`nepali-datetime`	1.0.8.5	Bikram Sambat conversion
Scraping	BeautifulSoup 4 / httpx	4.15 / 0.28.1	Notice-board parsing
Frontend framework	Next.js	16.3.1	App router, SSR, static build
UI library	React	19.2.8	Component rendering
Styling	Tailwind CSS	4.x	Design system
Markdown	`react-markdown` + `remark-gfm`	10.1 / 4.0	Answer rendering
Runtime (frontend)	Node.js / npm	22.23.1 / 10.9.8	Build and serve
Dependency manager	uv	0.12.1	Locked Python environment
Linting	Ruff / ESLint	0.16.3 / 9.x	Static analysis
Containerisation	Docker Compose	v2	Reproducible deployment

Two selections warrant justification. `qwen2.5:7b` [16] was selected because it fits within the eight gigabytes of video memory available on the development machine while

retaining usable instruction-following and multilingual competence; the consequences of that capacity limit are documented throughout Chapter 4. `bge-m3` [6] was selected for its explicit multilingual alignment, which permits an English query to match text derived from Nepali notices, and for its support of long inputs, which suits a chunking policy that preserves whole document sections.

3.5. Development Methodology

An iterative, issue-driven methodology was followed. Neither waterfall nor formal agile was appropriate: the requirement set could not be fixed in advance, because the failure modes that dominate this system's design were not predictable from its specification but were discovered by using it.

Work proceeded in cycles of the following shape:

1. **Observe.** A defect is reported, generally as a transcript in which the assistant answered wrongly.
2. **Reproduce and measure.** The behaviour is confirmed against the running system, and its extent quantified over a probe set before any change is made. Several reported defects proved on measurement to be worse, or differently caused, than reported.
3. **Diagnose.** The cause is located. In this project the reported symptom was repeatedly not the defect: a complaint of lost conversational memory proved to be an out-of-scope answer contaminating every subsequent turn, and a complaint of ineffective prompts proved to be the system prompt being silently discarded by the inference server.
4. **Implement.** The narrowest change that removes the cause is made, with the reasoning recorded as a comment at the site of the change.
5. **Re-measure.** The same probe set is re-run and the result recorded, including any regression the change introduced.
6. **Record.** The issue is closed with its measurements retained.

The project ledger records twenty-seven issues, of which twenty-one were closed over the development period, across thirty-eight commits. The measurements retained by that process constitute the evaluation reported in Chapter 5; they were taken as engineering evidence at the time each change was made, not reconstructed afterwards.

3.6. Data Handling

3.6.1. Acquisition

Fifteen source documents were obtained from the official notice boards: thirteen notices of the 2083 cycle – among them the information booklet [17], the result-publication notice and its pass list [18], the entrance syllabus [19] and the Pulchowk admission notice carrying the fee schedule [20] – together with the Pulchowk priority-applications list and the Pashchimanchal priority form. Each was downloaded in its published PDF form and retained unmodified, so that every derived artefact can be checked against its original.

3.6.2. Preparation

Each notice was converted to a Markdown document carrying YAML front matter with its title, publishing body, canonical notice URL, admission year, applicable level and, where relevant, its intended audience. Nepali documents were translated semantically rather than word-for-word, with key Nepali terms retained in parentheses on first use, Devanagari numerals converted to Arabic numerals and Bikram Sambat dates left unconverted so that the conversion machinery, not the translator, resolves them. Documents already in English – the syllabus and the four payment walkthroughs – were restructured for retrieval but not reworded. Each document records its own translation type, so a reader can tell reproduced text from translated text. Table 5 inventories the prepared corpus.

Table 5. Indexed corpus: thirteen translated official notices

#	Document	Chars	Chunks	Language
1	Quota documents for BE/BArch admission	14,867	20	Nepali
2	Foreign-student quota documents	3,572	4	Nepali
3	BE/BArch entrance result notice	7,096	8	Nepali
4	Urgent notice on application fee payment	3,874	5	Nepali
5	Instructions for online corrections	4,584	6	Mixed
6	Information and guidelines booklet 2083	67,310	84	Nepali
7	Detailed entrance syllabus 2083	16,361	19	English
8	Revised entrance examination notice	12,274	14	Nepali
9	Khalti payment walkthrough	6,532	10	English
10	connectIPS payment walkthrough	4,449	8	English
11	eSewa payment walkthrough	5,593	6	English
12	SBL BankSmart payment walkthrough	3,991	5	English
13	Pulchowk BE/BArch admission detail notice	20,899	27	Nepali
	Total	**171,402**	**216**	

Two tabular documents were transcribed to comma-separated values rather than prose, because their content is a table and embedding a table is useless. The published pass list yields 7,179 rows keyed by form number, each carrying merit rank, candidate name, district and remarks. The Pulchowk priority applications yield 1,700 rows, each carrying a merit rank, a quota group and up to nine ordered programme choices. Both are excluded from the vector index by directory convention and are read only by the exact-lookup modules.

3.6.3. Storage and Processing

Indexing splits each document first on its Markdown headings, preserving the document's own structure, then applies a recursive character splitter with a maximum chunk size of 1,200 characters and an overlap of 150 characters to any section that remains oversized. The resulting 216 chunks – mean length 781 characters, median 828 – are embedded and written to a persistent Chroma collection configured with a cosine metric over an HNSW index. Document front matter is attached to every chunk as scalar metadata, which is what later allows retrieval to be filtered by audience and citation to be grouped by source

document.

Three persistent stores are maintained: the vector index, a SQLite database holding both LangGraph conversation checkpoints and the thread index, and a JSON cache of scraped notices. Notice records are additionally written into the vector collection as short listings, so that a single retrieval covers both the corpus and the live feed, and so that the assistant can report the existence of a notice published after the corpus was prepared.

3.6.4. Input Handling and Privacy

Questions are normalised before the pipeline reads them: a request to answer in another language is removed, and a question written in Devanagari is translated to English. The student's original words are retained in the transcript; only the copy the model reads is rewritten. Exact lookups deliberately read the raw message rather than the normalised one, since a form number that survives a paraphrase may not survive a translation.

No account system exists. Conversations are scoped to a random identifier minted by the browser and held in local storage. This is not a security boundary and is not represented as one; it exists so that questions naming real candidates and their results do not appear in a stranger's history sidebar. All inference is local, so no question leaves the machine.

3.7. Testing and Validation

Validation was organised in five layers, summarised in Table 6.

Table 6. Validation strategy by layer

Layer	What is validated	Method
Self-verifying computation	Fee arithmetic against the figures printed in the notice	`verify()` re-derives all twenty published totals from line items; disagreement is an error, not a warning
Deterministic units	Date conversion, form-number and rank extraction, name and district matching, small-talk and scope pattern matching	Direct invocation over crafted inputs, including adversarial ones
Component behaviour	Retrieval relevance, reranking order, citation grounding	Scored probe sets of real questions; thresholds set from measured distributions rather than chosen
Integration	Full graph traversal end to end	Probe suites executed against the running service, counting correct answers, refusals and regressions
Interface	Streaming, history, scroll behaviour, responsive layout, theme	Manual exercise of the running application across breakpoints

Two aspects of this strategy are worth stating explicitly. First, every threshold in the retrieval and scope machinery was *measured* rather than chosen: the relevance floor, the scope-rescue threshold, the citation margin and the grounding fraction each sit inside a separation observed across a probe set, and Section 5.1 reports those separations. Second, regressions were treated as results. Where a fix introduced a new failure – as the scope guard did for bare conversational follow-ups – the regression was measured, recorded and repaired, and both measurements are reported.

3.8. Deployment Setup

The system supports two deployment modes.

Direct execution installs the Python environment from a lockfile using `uv`, builds the vector index once, scrapes the notice boards once, and runs the Uvicorn backend and the Next.js frontend as separate processes against a locally installed Ollama.

Containerised execution brings up the entire stack with a single Docker Compose invocation. Two images are built: a backend image based on a Python 3.12 image with `uv` preinstalled, and a frontend image producing a standalone Next.js build. An entrypoint

script builds the index and scrapes the notice board on first boot only, guarded by a generous health-check start period, and reuses both from named volumes thereafter. The `docs/` directory is bind-mounted so that documents uploaded through the administrative route persist into the repository, while the vector index and the notice cache occupy named volumes.

Ollama is deliberately *not* containerised in the default configuration. It remains on the host, because that is where previously pulled models reside and, on a machine with a graphics card, where the drivers are. On Linux this requires Ollama to be bound to all interfaces once, since a container cannot reach the host's loopback address; an alternative compose overlay is provided that runs Ollama in a container for hosts where that change is unwelcome.

Because the client calls the interface from the browser rather than from within the container network, the interface base address is compiled into the frontend bundle as a build argument rather than supplied as a run-time environment variable. Administrative routes are guarded by a bearer token supplied through the environment; if the token is absent, every administrative route returns HTTP 503 rather than running unauthenticated.

Chapter 4. Implementation Details

4.1. Requirements Specification

4.1.1. Functional Requirements

Table 7 states the functional requirements, each traced to the module that satisfies it.

Table 7. Functional requirements and implementing modules

ID	Requirement	Module
FR-1	Accept a question in English or Nepali and return a Markdown-rendered answer streamed token by token	`api.py`, `Markdown.tsx`
FR-2	Retrieve supporting passages from the indexed corpus and generate the answer conditioned upon them	`rag.py`, `graph.py`
FR-3	Name the documents an answer drew upon, linked to the official notices	`rag.py`
FR-4	Resolve a form number, merit rank, candidate name or district against the published pass list	`results.py`
FR-5	Compute every admission and study fee total for each of the four fee categories	`fees.py`
FR-6	Report last year's Pulchowk cut-offs for a stated merit rank and advise on priority ordering	`priority.py`
FR-7	Ground the current date and every Bikram Sambat date in both calendars, with day offsets	`dates.py`
FR-8	Aggregate notices from six official boards and extract dated obligations as deadlines	`notices.py`, `deadlines.py`
FR-9	Refuse out-of-domain questions in the application's own wording, and answer social messages without retrieval	`graph.py`
FR-10	Persist and title conversations, list them per browser, and restore a transcript with its citations	`threads.py`
FR-11	Permit an administrator to upload or delete a document, rebuild the index and refresh the notice feed	`api.py`

4.1.2. Non-Functional Requirements

Table 8. Non-functional requirements

ID	Attribute	Requirement
NFR-1	Correctness	Every figure a student may act upon shall be retrieved or computed, never generated. Fee totals shall be verified against the published totals at import time
NFR-2	Truthfulness	The system shall state when the corpus does not cover a question rather than answering from parametric memory
NFR-3	Attribution	A document shall be cited only where the answer visibly drew upon it
NFR-4	Privacy	All inference shall be local; no question shall leave the host machine
NFR-5	Responsiveness	The first token shall reach the client as soon as it is produced; refusals and social replies shall not incur generation cost
NFR-6	Portability	The stack shall build and run from a single Docker Compose invocation on Linux, macOS and Windows
NFR-7	Resource envelope	The system shall run within 8 GB of video memory and approximately 6 GB of model storage
NFR-8	Maintainability	Next year's cycle shall be accommodated by replacing documents and re-indexing, with no code change
NFR-9	Robustness	Failure of any single notice source, of the index, or of the model shall degrade that feature alone
NFR-10	Usability	The interface shall be legible on a mobile handset and shall support light and dark themes
NFR-11	Durability	A conversation shall survive a restart of the service

4.1.3. Constraints

Four constraints bounded the implementation. The generation model was limited to seven billion parameters by the eight gigabytes of video memory available. The corpus is fixed by what IOE publishes, which is scanned PDF and predominantly Nepali. The Bikram Sambat calendar admits no closed-form conversion and requires tabulated month lengths. And the deployment target is a single machine, not a cluster, so every component must share one process, one graphics card and one disk.

4.2. System Overview

The delivered system realises the methodology of Chapter 3 as follows. A question submitted from the client reaches `/api/chat`, which opens a Server-Sent Events stream and drives the compiled graph. The graph first decides whether the message requires documents at all. If it does, the question is normalised and condensed into a search query, passages are retrieved and reranked, the exact modules are consulted, and a scope guard decides whether the question is answered at all. Surviving questions reach the generation node, which assembles a prompt in a deliberate order, generates the answer, and filters the candidate citations against what the answer actually said. Tokens are forwarded to the client as they are produced; citations follow the final token as a separate event.

Table 9 maps each methodological commitment of Section 3.1 to the implementation that discharges it.

Table 9. Mapping from methodology to implementation

Methodological commitment	**Implementation**
Facts computed, not generated	`results.lookup_context`, `fees.fee_context`, `priority.priority_context` and `dates.annotate_dates` produce settled blocks that are injected into the prompt nearest the question
Translate at ingestion	Thirteen documents translated to English Markdown with provenance front matter; retrieval is monolingual
Explicit control flow	Conditional edges `route_question` and `route_scope` are predicates in code, not model instructions
Attribute after generation	`rag.keep_grounded` compares the finished answer against each candidate document's distinctive vocabulary
Refuse rather than approximate	`guard` and `refuse` emit fixed wording with no model call

4.3. Development Environment

Table 10. Hardware and software specification of the development environment

Item	Specification
Operating system	Linux, kernel 6.17
System memory	16 GB (8 GB minimum required for the 7B model)
Graphics memory	8.19 GB available to the model runtime
Model storage	Approximately 6 GB for `qwen2.5:7b` (4.7 GB) and `bge-m3` (1.2 GB)
Python toolchain	CPython 3.12.3, `uv` 0.12.1, locked with `uv.lock`
Node toolchain	Node.js 22.23.1, npm 10.9.8
Model runtime	Ollama 0.32.6, serving both models on port 11434
Editor and analysis	Ruff for Python, ESLint with the Next.js configuration for TypeScript
Version control	Git; 38 commits over the development period
Containers	Docker Compose v2, two services plus an optional Ollama overlay

Python dependencies are resolved from a lockfile so that the environment is reproducible; the container image installs dependencies before the source is copied, so that editing the source does not re-resolve them.

4.4. System Implementation

4.4.1. Ingestion and Indexing

Indexing walks the document directory, skipping the untranslated source PDFs, the tabular lookup data and internal engineering notes by directory convention. Each document's YAML front matter is separated from its body, and only populated scalar values are retained as metadata, since the vector store rejects null and non-scalar metadata.

Splitting proceeds in two stages. A header-aware splitter divides the body on its first-, second- and third-level Markdown headings, retaining the headings in the text so that each chunk carries its own context. Any section still exceeding the maximum size is divided again by a recursive character splitter with $s = 1200$ characters and $o = 150$ characters of overlap, giving the chunk count of Equation (5). Applied to the corpus of Table 5, this produces 216 chunks with a mean length of 781 characters and a median of 828, the largest being 1,199. Figure 3 shows the pipeline.

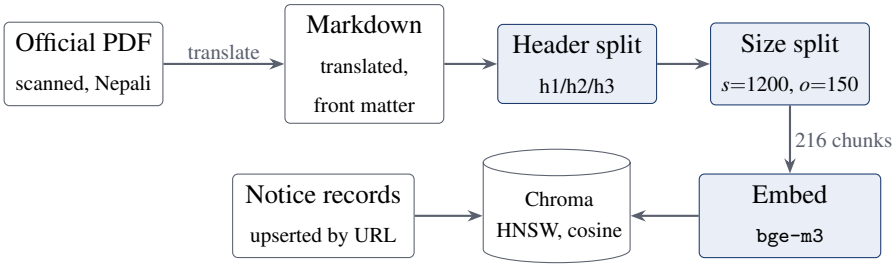

Figure 3. Document ingestion and indexing pipeline

Notice listings are written into the same collection as the documents, keyed by notice URL so that writing them is an upsert and a notice that has rolled off the feed can be deleted by identifier. Keeping them in one collection means a single retrieval covers both the corpus and the live feed, and the citation machinery treats a notice exactly as it treats a document. A full rebuild resets the collection and then re-inserts the notice records, so that a rebuild never leaves the assistant blind to the feed.

4.4.2. Retrieval, Thresholding and Reranking

Retrieval over-fetches, requesting $2k$ candidates for a target of $k = 6$, so that a demoted passage can genuinely be displaced by a better-suited one rather than merely reordered among those already selected.

Reranking then applies a domain rule. The corpus contains documents addressed exclusively to foreign applicants, who are a small minority of the population asking. Unless the query itself signals a foreign applicant, such passages are demoted by a fixed penalty:

$$s'(q,d) \;=\; \begin{cases} r(q,d) - \lambda, & \text{if } \mathrm{aud}(d) = \text{foreign and } \neg\,\phi(q), \\ r(q,d), & \text{otherwise,} \end{cases} \tag{6}$$

where $s'(q,d)$ is the adjusted score of passage d for query q, $r(q,d)$ the relevance of Equation (3), $\mathrm{aud}(d)$ the audience recorded in the passage metadata, $\phi(q)$ a predicate true when the query mentions foreign or international applicants, and $\lambda = 0.10$ the penalty. The value of λ was chosen against measurement: the spread between competing passages on a quota question is approximately 0.03, so λ reliably reorders, while remaining small enough that a foreign-only document still surfaces when nothing else matches.

The retained set is then the best k passages clearing a relevance floor:

$$\mathscr{R}(q) \;=\; \text{top-}k\,\big\{\, d \in \mathscr{D} \;:\; s'(q,d) \geq \tau \,\big\}, \qquad k = 6,\ \tau = 0.45, \tag{7}$$

where $\mathscr{D}$ is the indexed passage set, τ the relevance floor and k the number of passages

admitted to the prompt. The floor is set well below the observed on-topic band, since its purpose is to exclude unrelated passages when a question misses the corpus entirely, not to arbitrate among plausible ones; measured separation on this corpus is wide, at approximately 0.6 for on-topic passages against 0.3 for off-topic ones.

4.4.3. Query Normalisation and Rewriting

Before retrieval, the question is put into the form the model should read. Two transformations are applied, both deterministic in their trigger.

A request to answer in another language is removed from the question. The test requires a language name *and* a speech verb in the same sentence, so that "is the examination set in Nepali?" – a question about the examination – survives untouched; and if removal would leave nothing, nothing is removed, so that "can I answer the paper in Nepali?" is treated as the ordinary question it is. A fixed sentence written by the application, not by the model, is then prefixed to the reply. Section 4.7 records why the request is removed rather than refused.

A question written predominantly in Devanagari is translated to English before the model reads it. Translation removes the language the model would otherwise mirror, and retrieval gains from it as well, since an English query embeds against an English index rather than across languages. The student's own words remain in the transcript; only the copy the model reads is rewritten.

Finally, on a follow-up turn, the question is condensed against the last six messages into a standalone search query. The first turn requires no condensing.

4.4.4. Small-Talk Routing

A message that is socially complete in itself – a greeting, a thank-you, an acknowledgement – is routed directly to generation, bypassing rewriting and retrieval entirely. The test is a whole-message match against a fixed vocabulary rather than a judgement, so anything with a question attached fails it and takes the ordinary path. This asymmetry is deliberate: skipping retrieval on a real question costs an ungrounded answer, while not skipping it on a greeting costs approximately one second.

The measured cost that motivated this is instructive. Because query rewriting is instructed to resolve implied subjects from the conversation, and a greeting has no subject, "thanks" was being rewritten into the previous turn's topic, which then retrieved several kilobytes of context that the reply "you're welcome" could not use – approximately 0.7 s in the rewrite call and 0.5 s in the embedding call, and a longer prompt for every turn thereafter. Three tokens are deliberately excluded from the vocabulary – "yes", "no" and "sure" – because they are frequently the answer to a question the assistant asked, and that turn

may well need the documents.

4.4.5. The Scope Guard

The guard decides whether a question is answered at all. It combines three signals, and refusal requires all three to fail, because refusing a genuine question is the more damaging error:

$$\text{Refuse}(q) \;=\; \neg\Big[\; \underbrace{L(q)}_{\text{exact hit}} \;\vee\; \underbrace{C(q,h)}_{\text{classifier}} \;\vee\; \underbrace{\big(m(q) \geq \sigma\big)}_{\text{relevance rescue}} \;\Big], \qquad \sigma = 0.60, \tag{8}$$

where $L(q)$ is true when the question produced an exact pass-list, fee or priority block; $C(q,h)$ is the verdict of a yes/no classifier applied to the question q shown alongside the recent transcript h; $m(q)$ is the best relevance attained by the *raw* question against the index; and σ is the rescue threshold.

Three implementation details carry the design. The classifier is framed as plausibility – "could this be about IOE? When in doubt, say yes" – rather than as a binary in/out decision, a framing difference worth ten points of accuracy in measurement (Section 5.1). The relevance signal is computed against the student's own words rather than the rewritten query, because the rewrite is what makes a follow-up IOE-shaped and a signal the pipeline itself contaminated cannot judge whether the input was in scope. And the classifier is shown the last two turns verbatim, because a bare follow-up such as "how many are there?" carries no topic of its own and is refused if judged alone.

A refusal emits fixed wording with no model call, and no citations, because nothing was read. The refusal is written to the conversation state so that a reloaded transcript shows the same sentence rather than an empty turn.

4.4.6. Exact Lookup Over the Pass List

The published pass list is a 7,179-row table keyed by form number. It is deliberately excluded from the vector index: similarity search over names and roll numbers returns near-noise, and a student asking whether a particular form number passed requires an exact record, not a nearest neighbour.

Four lookup modes are supported. *Form number* and *merit rank* are keys into exact indices and are unambiguous. *Name* is not: Nepali names run to two, three or four words with no reliable division into given and family names, and a single common surname matches hundreds of rows. The matching vocabulary is therefore built from the published list itself – a phrase or word matches only if a real candidate actually bears it – with longest-phrase-first consumption so that a full name is read as one person rather

than as a full-name match plus a stray surname hit. Where two or more name words are asked about together, the intersection is reported rather than each word's matches separately, since the question is whether one candidate carries both. *District* is gated: a district named without any name alongside it counts as a pass-list question only if the message also carries result-seeking vocabulary, since this domain discusses quotas and campuses geographically all the time.

Every rendered block is terse labelled fields rather than prose. A fluent paragraph placed in the prompt is copied to the student verbatim, guidance and all; fields oblige the model to write the answer itself. The blocks encode three distinctions the model cannot be trusted to make: a name that matched a different district is reported as exactly that and never as an absence; multiple matches are listed with an explicit instruction not to choose among them; and a capped district listing states that it is a ranked subset.

4.4.7. Deterministic Fee Computation

The fee tables are in the corpus and retrieval finds them, yet the model answered fee questions wrongly because reading them requires arithmetic: a per-semester subtotal multiplied by eight, three tables summed, and one figure – the amount due on admission day – that is a *part* of the degree total rather than an addition to it.

Only line items are stored. Every total is summed from them. For fee category c, with per-semester items A_c, one-time items B_c and refundable deposits D_c,

$$P_c = \sum_{i \in A_c} a_i, \qquad O_c = \sum_{j \in B_c} b_j, \qquad G_c = \sum_{k \in D_c} d_k, \tag{9}$$

$$T_c^{\text{adm}} = P_c + O_c + G_c + I + C, \tag{10}$$

$$T_c^{\text{deg}} = N\,(P_c + I) + O_c + G_c + C, \tag{11}$$

$$T_c^{\text{net}} = T_c^{\text{deg}} - G_c, \tag{12}$$

where a_i, b_j and d_k are individual published line items; P_c, O_c and G_c are the per-semester, one-time and deposit subtotals; $I = 600$ is the per-semester internet charge and $C = 1000$ the one-time engineering council charge, both levied outside the three tables; $N = 8$ is the number of semesters in the BE programme (ten for BArch); T_c^{adm} is the amount due on admission day; T_c^{deg} the total across the degree; and T_c^{net} that total net of refundable deposits.

A verification routine re-derives all five published totals for each of the four categories from the line items and reports disagreement. This is the mechanism by which a transcription error fails loudly instead of silently teaching the assistant a wrong number,

and it is also how a conflict between two source documents was adjudicated: where the campus notice and the booklet disagree, the notice is preferred, because its totals all reproduce from its own line items and the booklet's do not.

The rendered block deliberately shows no working. An earlier version printed its arithmetic, and the model copied the habit rather than the result, producing wrong sums around a right figure. Order and labelling within the block carry more weight than the instructions above it: the totals are placed first because the model answers with the first plausible row it meets, the phrase "at admission" appears on the total line and nowhere else, and the three individual deposits are given as prose beneath the table rather than as rows, because as rows they outcompeted their own total.

4.4.8. Reconstruction of Programme Cut-offs

IOE assigns Pulchowk seats by serial dictatorship, as established in Section 2.1.7. Algorithm 1 reconstructs the cut-offs from the published inputs.

Table 11. Algorithm 1: cut-off reconstruction by serial dictatorship

Input: applicants $\mathscr{A}$ sorted by merit rank; for each $a \in \mathscr{A}$ an ordered priority list π_a of programme codes; seat capacity κ_p for each programme p
Output: cut-off γ_p, the merit rank of the last candidate admitted to p

```
for each p: f_p ← 0; γ_p ← ∅
for each a ∈ 𝒜 in ascending rank order
  for each p ∈ π_a in declared order
    if f_p < κ_p then
      f_p ← f_p + 1;  γ_p ← rank(a);  break
return γ
```

A candidate who applied under more than one quota appears on more than one row; the open-quota row governs open-seat competition and wins the deduplication, reducing 1,700 rows to 1,647 distinct applicants. Seat capacities are taken from section 2.7 of the published booklet. The resulting cut-offs are reported in Section 5.1.

A rank ρ is then said to clear programme p if

$$\rho \le \gamma_p, \qquad \gamma_p \neq \varnothing, \tag{13}$$

where ρ is the student's merit rank and γ_p the reconstructed last-admitted rank. The verdict, in words, is welded to every line of the rendered block rather than left for the

model to derive, because a seven-billion-parameter model handed a bare pair of numbers compares them unreliably. The block also states the ordering strategy outright – declare programmes in genuine order of preference, and place a Regular seat above its Full-fee counterpart – and forbids the plausible-sounding tactics that strategy-proofness makes futile.

4.4.9. Calendar Grounding

Two blocks are prepared before generation. The first states today's date in both calendars, anchored to Nepal Standard Time so that "today" is correct for the student regardless of where the service runs. The second resolves every Bikram Sambat date appearing in the question or the retrieved passages to its Gregorian equivalent and to a plain-language offset:

$$\Delta(t) \;=\; \mathrm{AD}(t) - \mathrm{AD}(t_0), \tag{14}$$

where t is a Bikram Sambat date found in the text, t_0 is today in Bikram Sambat, $\mathrm{AD}(\cdot)$ is the tabulated conversion, and Δ is the signed offset in days, rendered as "in n days" or "n days ago". The model is instructed to read these off and told explicitly not to perform calendar arithmetic. Out-of-range dates printed in a document are skipped rather than guessed.

The same conversion serves the deadline list. Every Bikram Sambat date in the corpus is located, the Markdown line containing it is taken as its context, and the date is retained as a deadline only if that line reads like an obligation and not like document metadata. Retained entries are windowed to 45 days past and 365 days ahead, deduplicated by document and date, and sorted. Inline Markdown is stripped from the extracted snippet, since it is displayed as plain text.

4.4.10. Prompt Assembly and Citation Filtering

The generation node assembles its prompt in a deliberate order: system instruction, today's date, the notice digest, date conversions, the pass-list block, the retrieved documents, and finally the computed fee and priority blocks. The last placement is not stylistic. Retrieval routinely puts the raw fee tables and the booklet's seat totals into the same prompt, and with a model of this capacity whatever sits nearest the question wins; the settled figures therefore go nearest the question, so that they beat the raw material they were computed from.

Citations are assembled from what retrieval placed in the prompt and then narrowed to what the finished answer visibly used. Candidates are first grouped by source document,

one citation per document, and any document more than a small margin below the best hit is dropped:

$$\mathscr{C}(q) \;=\; \big\{\, d \;:\; s'_{\max} - s'(q,d) \le \delta \,\big\}, \qquad \delta = 0.03, \tag{15}$$

where $s'_{\max}$ is the best adjusted score attained and δ the citation margin. Questions that genuinely span several notices retain all of them; questions one notice answers outright cite that one alone.

The surviving candidates are then tested against the answer. Let W be the set of distinctive terms in the finished answer with the question's own terms removed, and S_d the corresponding set for document d. Document d is cited if

$$|W \cap S_d| \;\ge\; \mu \quad \text{and} \quad \frac{|W \cap S_d|}{|W|} \;\ge\; \nu, \qquad \mu = 4,\ \nu = 0.16, \tag{16}$$

where μ is the minimum number of shared distinctive terms and ν the minimum share of the answer's own vocabulary they must constitute. Terms supplied by the question are discounted on both sides, since a document earns no citation by echoing the words it was searched with, and a stop list removes both ordinary English function words and the domain vocabulary common to every notice in the corpus.

The thresholds were measured, not chosen. Across eighteen real answers, a grounded answer shares between 0.17 and 0.75 of its own vocabulary with its source, while a refusal shares between 0.02 and 0.15; the cut sits in that band and errs downward, because a lost citation costs a link the student can still find in the notice rail, whereas an invented one places a source under a sentence it never wrote. The count threshold guards the other end, where a very short answer sharing one incidental term would otherwise pass on fraction alone.

Three blocks bypass this filter. Where an exact pass-list, fee or priority block was used, the corresponding notice is cited outright and takes the leading slot, because the figures came from that notice's own tables whatever else retrieval happened to supply. Citations are attached to the message rather than to the conversation state, so that they remain with the turn they belong to and are replayed correctly when a transcript is reloaded.

4.4.11. Notice Aggregation

Six official notice boards are scraped: the Entrance Examination Board, the Institute of Engineering, Tribhuvan University, and the Pulchowk, Pashchimanchal and Purwanchal campuses. Each has its own parser, deliberately small and defensive, so that one site changing its markup or going down degrades that source to an error entry and leaves the

rest intact. Listings are deduplicated by URL, dated in both calendars and cached as JSON.

Notice *pages* are not fetched. Sampling one page from each source found each to be a heading above a link to a scanned PDF – the richest held 209 characters, most of it navigational – so what is worth indexing is the record itself: what was published, by whom, and when. The prompt digest accordingly instructs the model to report a notice's existence and date and to link to it, and never to describe its contents on the strength of its title.

Two campuses are deliberately absent. Thapathali renders its notice list in the browser, so its served markup contains no notices and scraping it would require a headless browser; Chitwan publishes no feed of its own and links to Tribhuvan University, which is already a source.

4.5. Interface and Visualization

The client is an editorial rather than a conversational interface: the assistant is presented as a publication with a masthead, a dateline in both calendars and a notice rail, rather than as a chat window. The figures in this section are screenshots of the running system, captured against the deployed stack described in Section 4.6.

Figure 4 shows the landing route. The dateline states the current date in both calendars, and the three claims beneath the fold are the only place the application describes itself rather than reporting a notice.

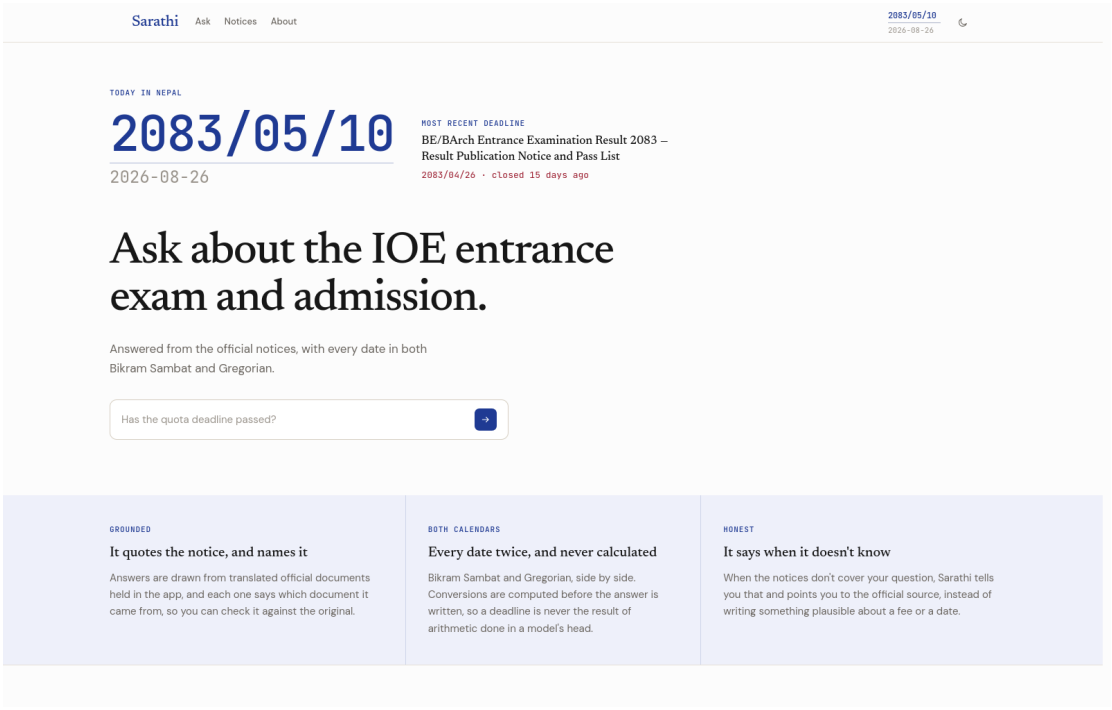

Figure 4. Landing route, with the dual-calendar dateline and the composer

Figure 5 shows the three-column workspace at the `/ask` route answering a retrieval question. Conversation history occupies the left column, the transcript and composer the centre, and the live notice rail the right. The answer is rendered from Markdown – nested lists, bold emphasis, and Nepali terms quoted parenthetically after their English rendering – and the citation block beneath it names the two notices the answer drew upon, each with the specific sections retrieved and a link to the published original. The conversation title in the sidebar was written by the model from the opening question.

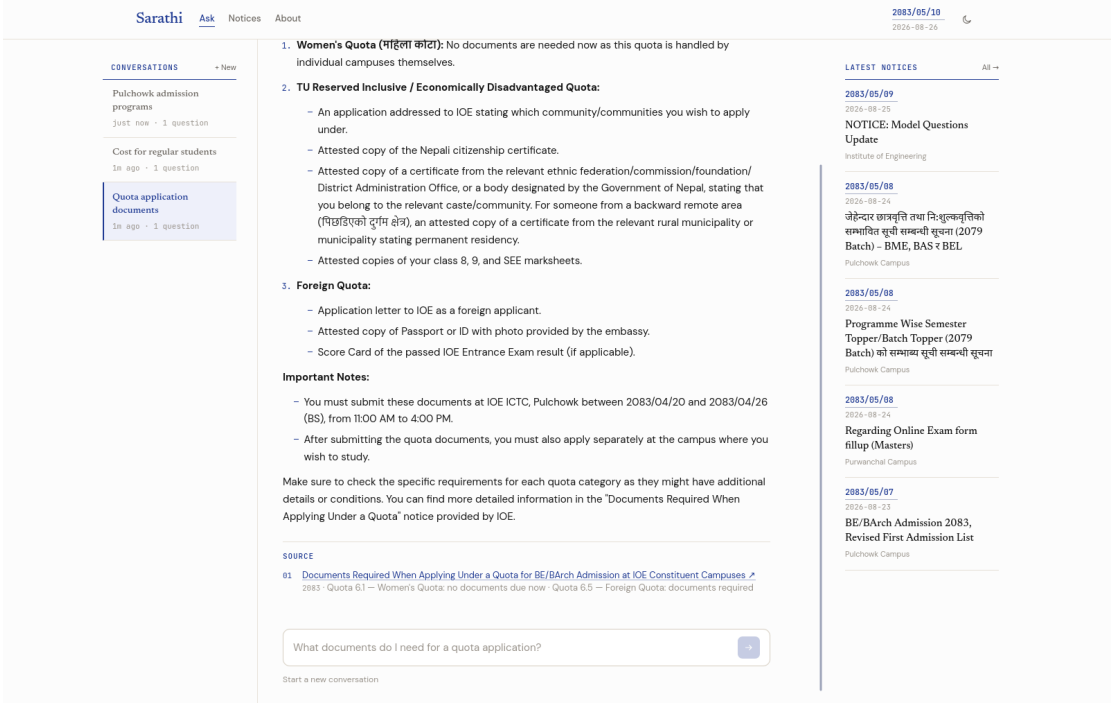

Figure 5. Grounded retrieval answer with per-document citations at `/ask`

Figure 6 shows the same interface answering a fee question, which is served by the deterministic arithmetic of Section 4.4.7 rather than by generation. Every figure in the answer is read from the computed block, and the notice whose tables the figures were derived from is cited outright.

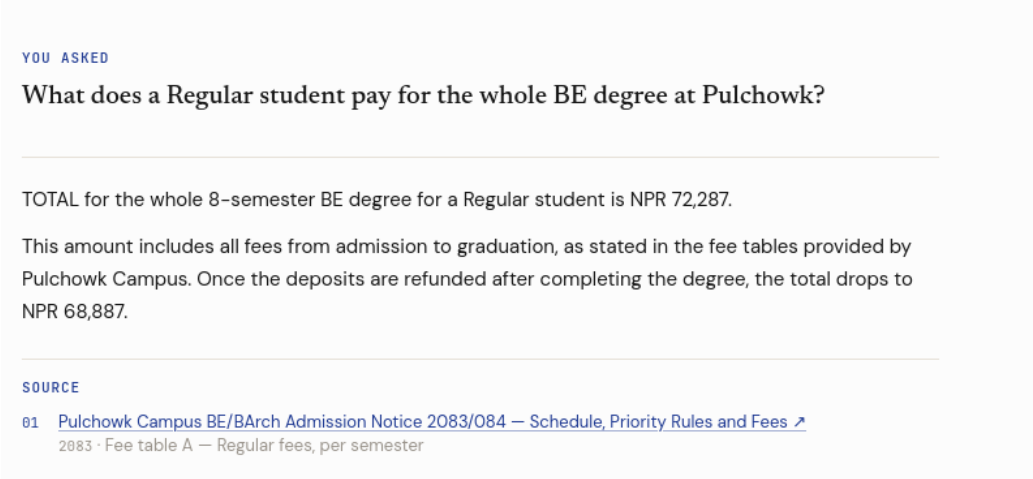

Figure 6. Fee answer produced from computed totals rather than generated arithmetic (transcript column only)

Figure 7 shows the aggregated notice feed at `/notices`, collecting six official notice boards into a single index, newest first, with each publication date given in both calendars and attributed to its publisher.

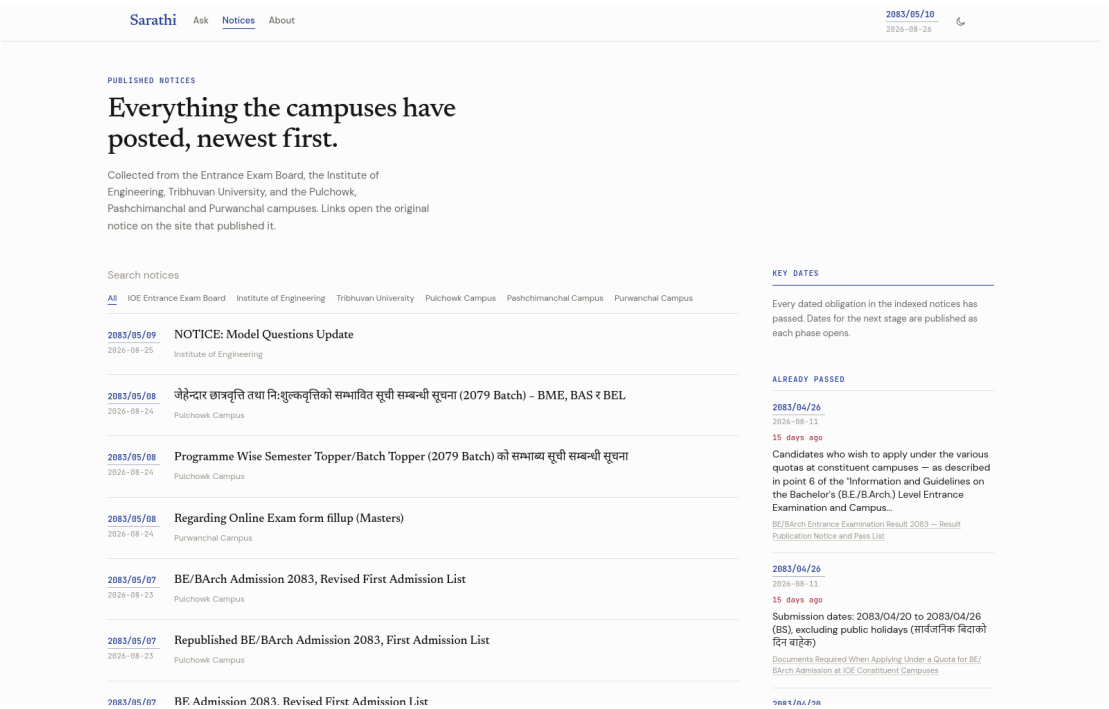

Figure 7. Aggregated notice feed at `/notices`, six sources in one index

Above the extra-large breakpoint the page itself does not scroll; the three columns scroll independently, so that reading a long answer never carries the notice rail off the screen. Below that breakpoint the layout collapses to one ordinary scrolling page with the history behind a toggle, since a fixed-height shell on a handset yields a conversation column a few centimetres tall.

Answers are streamed and rendered as Markdown, supporting paragraphs, bullet and numbered lists, emphasis, headings, tables and links. Citations appear beneath each answer as named sources linking to the official notice. Three behaviours of the streaming view were implemented in response to observed defects: the view follows the answer only while the reader is already at the foot of the transcript, so that scrolling up mid-answer is not undone; the transcript reserves space for the composer, so that the final lines are not hidden beneath it; and a page refresh does not re-submit the question that opened the conversation.

4.6. Deployment and Execution

Direct execution requires four steps: install dependencies from the lockfile with `uv sync`; pull the two models with Ollama; build the index with `uv run ioe-index`; and start the backend and frontend. The notice board is scraped with `uv run ioe-notices` and refreshed automatically thereafter whenever the cached feed exceeds twenty minutes in age, the refresh running behind the answer rather than delaying it.

Containerised execution reduces this to `docker compose up -build`. The backend entrypoint builds the index and scrapes the notice board on first boot, which takes several minutes and is accommodated by a health check with a 300-second start period; subsequent boots reuse both from named volumes. The frontend is served on port 3000 and the backend on port 8000, both configurable.

Administrative routes permit a document to be uploaded or deleted, the index to be rebuilt and the notice feed to be refreshed. Uploads are constrained to Markdown files with safe names below a size limit, and every administrative route is guarded by a bearer token; where no token is configured, the routes return HTTP 503 rather than executing unauthenticated. The container runs as an unprivileged user whose identifier matches the usual first Linux account, so that documents uploaded into the bind-mounted directory remain editable on the host.

4.7. Challenges and Solutions

This section records the principal technical challenges encountered, each with the diagnosis that resolved it. They are reported because in every case the reported symptom differed materially from the cause, and because the resulting design decisions are otherwise inexplicable.

4.7.1. The System Prompt Was Being Discarded

Symptom. The prompts were reported as long and ineffective. Rules accumulated over twenty-three issues appeared to have no effect, and the assistant invented programmes that IOE does not offer.

Diagnosis. The model client was constructed without specifying a context window, so the runtime served the model at its own default of 4,096 tokens, confirmed on the running server. A representative grounded fee question assembles approximately 5,318 tokens before any conversation is added: 2,131 for the system instruction, 1,656 for six retrieved passages, 1,161 for the computed fee figures, 315 for the notice digest and 52 for the date blocks. Content exceeding the window is discarded silently, and the system instruction, being first, is discarded first. This was confirmed directly rather than inferred: a probe prompt of 8,040 tokens evaluated only 2,050 at the default window and answered from filler at the end rather than from the instruction at the start, while the same prompt at 8,192 evaluated 8,037 and answered correctly.

Solution. A single context-window constant of 16,384 tokens was defined beside the runtime address and applied to *every* model client in the process. The uniformity is not tidiness: the window is a load-time option, so two clients disagreeing about it cause the runtime to hold two model instances, and two do not fit on an eight-gigabyte card. The larger of the two viable settings was chosen despite its cost – 5.47 GB of 8.19 GB against 4.99 GB at 8,192 – because the worst-case prompt leaves too little slack against the smaller.

4.7.2. Answers in the Wrong Language

Symptom. Asked to reply in Nepali, the assistant replied in a mixture of Hindi and Nepali; asked a question in Nepali with no instruction at all, it answered in Nepali and invented syllabus content while doing so.

Diagnosis. The rule existed in the system instruction and the model did not follow it. Four successive prompt formulations were attempted and each failed differently, as recorded in Table 16. The pattern common to all four is that *naming* the request in the prompt is what keeps it alive.

Solution. The request is not named; it is removed. The sentence asking for another language is stripped from the question, the application emits its own fixed sentence explaining that it answers only in English, and the model receives an ordinary question – which it answers in English, because that is what it does with ordinary questions. A question written in Devanagari is translated to English before the model reads it, removing the language being mirrored. The nine-line language paragraph was deleted

from the system instruction at the same time, leaving one sentence.

4.7.3. Out-of-Scope Answers Corrupting the Conversation

Symptom. Reported as two defects: the assistant answered a question about holidays with a six-point planning questionnaire, and then appeared to have lost its memory of the conversation.

Diagnosis. One cause, not two. Conversational memory was intact and verifiably so. What breaks the conversation is the off-scope answer itself: once a holiday questionnaire is in the transcript, the follow-up "bahamas" is retrieved against and reasoned over as though it were an IOE question, and every turn thereafter is incoherent. The apparent memory loss was the symptom; answering the question was the defect. No scope check existed anywhere in the graph – the system instruction asked the model to stay on topic, and with six kilobytes of retrieved context before it, it did not.

Solution. The check was made a node of the graph rather than an instruction in a prompt, with the three-signal structure of Equation (8). Two findings shaped it, both from measurement rather than reasoning. Retrieval relevance alone cannot decide scope, because the distributions overlap: in-scope questions scored 0.435 to 0.700 and off-scope questions 0.357 to 0.573, and "how do I apply to Kathmandu University" at 0.573 outranks "how many seats are there in Pulchowk" at 0.478. And the signal must be read from the student's own words: an early version read relevance from the rewritten query, and the rewrite had turned "lets go on a holiday" into "IOE admission documents for holiday related scholarships" at 0.615, which sailed through the rescue.

4.7.4. Arithmetic Performed Inside the Model

Symptom. Asked what a Regular student pays for the whole degree, the assistant multiplied correctly and then trailed off; asked for a total, it invented a per-programme tuition table appearing in no document; asked what is refundable, it invented a refund policy.

Diagnosis. The fee tables were being retrieved correctly. The failure was arithmetic, and arithmetic across three tables with one figure that is a part of the total rather than an addition to it.

Solution. The totals are computed in code and supplied already worked out, with a verification routine checking each against the printed figure. Placement then proved to matter as much as computation: positioned ahead of the retrieved documents the computed block was ignored, because the raw tables sat nearer the question. Moving it after the documents fixed it. The internal ordering of the block matters similarly, as described in Section 4.4.7.

4.7.5. Citations Beneath Answers That Did Not Use Them

Symptom. A refusal was printed above a list of sources, and answers cited every passage retrieval had supplied rather than those they had used.

Diagnosis. Citations were being assembled from the retrieval result, so they described what was *available* rather than what was *used*. No relevance score can distinguish the two: "how do I apply to Kathmandu University" scores 0.63 against this corpus, higher than several genuine questions, and the honest response to it is a refusal.

Solution. The lexical grounding test of Equation (16), applied after generation. Requesting citation markers from the model was considered and rejected: a model of this capacity does not emit them reliably, and a citation that is sometimes present is worse than none, because the student cannot distinguish a missing marker from an ungrounded answer.

4.7.6. Invisibility of Notices Newer Than the Corpus

Symptom. Asked whether anything had been published recently, the assistant said no, while the interface displayed a notice from the previous day.

Diagnosis. The notice cache was not visible to the assistant at all. It was scraped, cached and rendered by the client, but never entered the prompt.

Solution. Notice listings are indexed as short records in the same collection as the documents, and a digest of the most recent notices is injected into every prompt, not only into prompts that appear time-sensitive – since a question need not mention notices to be answered wrongly in this way. The digest carries a strict instruction that a title is not contents: the assistant may report what was published and when, and link to it, but may not describe what it says.

4.7.7. Latency Spent on Messages That Needed None

Symptom. A one-word acknowledgement cost as much as a substantive question and made every subsequent turn slower.

Diagnosis. Query rewriting resolves implied subjects from the conversation, and a greeting has no subject, so it borrowed the previous one.

Solution. The small-talk route of Section 4.4.4, with the deliberate asymmetry described there.

Chapter 5. Results and Discussion

5.1. Results

The results reported here were obtained as engineering evidence during development, each measurement taken against the failure mode the corresponding change was made to remove. They are presented in the order the system encounters them: corpus and index, retrieval behaviour, scope adjudication, language handling, prompt capacity, exact computation, and delivered functionality.

5.1.1. Corpus and Index

Ingestion of the thirteen prepared documents produced 216 chunks from 171,402 characters of English text. Table 12 summarises the resulting index; the per-document distribution appears in Table 5, and is strongly uneven, with the 32-page information booklet alone contributing 84 chunks, or 38.9% of the index.

Table 12. Index statistics after ingestion

Property	**Value**
Source documents indexed	13
Total indexed text	171,402 characters
Chunks produced	216
Mean chunk length	781 characters
Median chunk length	828 characters
Largest chunk	1,199 characters
Configured maximum chunk size	1,200 characters
Configured overlap	150 characters
Largest single contributor	Booklet 2083, 84 chunks (38.9%)
Excluded from the index (by design)	2 tabular files, 8,879 rows
Vector metric / index structure	Cosine / HNSW

That the largest chunk falls one character below the configured maximum indicates that the header-aware first stage, not the size-bounded second stage, governs most divisions – which is the intended behaviour, since document sections are more coherent retrieval units than fixed-width windows.

5.1.2. Retrieval and Citation Thresholds

Every threshold in the retrieval path was set from a measured separation rather than chosen. Table 13 states each threshold, the separation observed, and the consequence of the setting.

Table 13. Measured separations underlying each retrieval threshold

Threshold	Value	Measured separation
Relevance floor τ	0.45	On-topic passages score approximately 0.6 against approximately 0.3 off topic; the floor sits between and excludes unrelated passages when a question misses the corpus
Foreign penalty λ	0.10	Competing passages on a quota question are separated by approximately 0.03, so the penalty reliably reorders while remaining small enough for a foreign-only document to surface unopposed
Citation margin δ	0.03	A question spanning three notices produced scores within 0.02 of one another, all retained; a question one notice answers outright cites that one alone
Shared terms μ	4	Guards short answers, where a single incidental term would otherwise satisfy the fraction test alone
Shared fraction ν	0.16	Over eighteen real answers, grounded answers share 0.17–0.75 of their vocabulary with their source; refusals share 0.02–0.15
Scope rescue σ	0.60	In-scope questions score 0.435–0.700, off-scope 0.357–0.573; the threshold sits above the off-scope maximum

The grounding separation is the most consequential of these. The bands 0.17–0.75 and 0.02–0.15 do not overlap, which is what makes a purely lexical test viable in place of the trained or sampled methods reviewed in Section 2.2.3; the threshold is placed within the gap and biased downward, since a lost citation costs a link the student can recover from the notice rail whereas an invented one misattributes a sentence.

5.1.3. Scope Adjudication

The scope classifier's framing was found to dominate its accuracy. The same model, over the same 39-question probe set, differed by ten points between framings, as Table 14 shows.

Table 14. Effect of classifier framing on scope accuracy (39-question probe set)

Framing of the classification prompt	Score	Error profile
"IN or OUT of scope"	26/39	17/17 off-topic correct, but only 9/22 genuine questions accepted
"ANSWER or REFUSE"	32/39	Intermediate
"Could this plausibly be about IOE? When in doubt, say YES"	**36/39**	Remaining errors are genuine questions with unfamiliar vocabulary, which the relevance rescue recovers

The first framing is instructive as a negative result. Its error profile is exactly inverted with respect to the cost structure of the application: a student turned away with a genuine question has no recourse, whereas an off-topic answer costs a few wasted seconds. The adopted framing biases toward acceptance and delegates the residual risk to the two other signals.

Adding the transcript to the classifier's input repaired a regression the guard had introduced for bare conversational follow-ups. On a 27-case matrix constructed for that purpose, the final configuration scored 26/27, the single failure being a payment question that the relevance rescue independently covers. Table 15 reports live verification against the running system.

Table 15. Live verification of the scope guard (32 checks)

Case class	Result	Cost
Off-topic questions refused	10/10	≈1.5 s, no model generation
Genuine questions answered	10/10	Ordinary path
Bare conversational follow-ups answered	8/8	Ordinary path
Small talk answered	Under 1 s, retrieval bypassed	
Originally reported transcript	Both off-topic turns refused; subsequent genuine question answered with citation	

5.1.4. Language Handling

Four successive prompt-based attempts to enforce English failed, each in a different way. Table 16 records them alongside the deterministic approach that replaced them.

Table 16. Enforcing English: prompt-based attempts against the deterministic solution

Approach	Result
Rule as one bullet within the system instruction	0/4 responses in English
Hoisted `Language:` section, "refuse then answer"	The refusal itself was written in Nepali
"Copy this exact sentence, then answer"	12/12 in English, but 3/12 sent the sentence alone as the entire reply
"The refusal is handled; ignore the request"	6/18 in English – a regression
Deterministic removal plus ingestion-side translation	**24/24 turns in English** across 8 cases run 3 times each; zero Devanagari in every case that requested another language or was written in one

The residual 1–3% Devanagari observed in the English control cases is the desirable kind: official Nepali terms quoted parenthetically after their English rendering, a convention the source notices themselves employ.

5.1.5. Prompt Capacity

Table 17 gives the token budget of a representative grounded fee question on its first turn, measured against the served context window before the defect described in Section 4.7.1 was corrected.

Table 17. Token budget of a representative grounded prompt, first turn

Prompt component	Tokens
System instruction	2,131
Retrieved documents (6 passages)	1,656
Computed fee figures	1,161
Notice digest	315
Today's date and conversions	52
Total, first turn, no history	**5,318**
Context window served by default	4,096
Context window after correction	16,384

The consequence was verified directly rather than inferred. A probe prompt of 8,040 tokens, comprising an instruction followed by filler followed by the question, evaluated 2,050 tokens at the default window and answered from the filler; the identical prompt at 8,192 evaluated 8,037 tokens and answered correctly. Table 18 states the resource cost of the correction.

Table 18. Video memory occupancy against context window

Context window	Occupancy	Available	Disposition
4,096 (default)	—	8.19 GB	Prompt truncated
8,192	4.99 GB	8.19 GB	Insufficient slack
16,384 (adopted)	5.47 GB	8.19 GB	Adopted

After the correction, the running server reported one resident model at the configured window, confirming that the single shared constant had prevented the runtime from holding two model instances.

5.1.6. Fee Computation

The verification routine re-derives five published totals for each of the four fee categories – twenty checks in all – from the stored line items. All twenty reproduce exactly; the routine returns no discrepancies. Table 19 reports the derived subtotals against the printed figures, and Table 20 the totals a fee question can require.

Table 19. Verification of published fee subtotals against derived sums (NPR)

Subtotal	Regular	Full Fee	Foreign	Sponsored
Per semester (Table A)	6,974	52,766	147,167	66,822
One-time (Table B)	7,295	123,704	273,435	123,705
Deposits (Table C)	3,400	40,000	115,000	40,000
Three tables at admission	17,669	216,470	535,602	230,527
Total due at admission	19,269	218,070	537,202	232,127
Discrepancies against printed totals	**0**	**0**	**0**	**0**

Table 20. Computed fee totals across the eight-semester BE programme (NPR)

Total	Regular	Full Fee	Foreign	Sponsored
Due on admission day	19,269	218,070	537,202	232,127
Whole degree, gross	72,287	591,632	1,571,571	704,081
Whole degree, net of deposits	68,887	551,632	1,456,571	664,081
Refundable deposits	3,400	40,000	115,000	40,000

Verification also adjudicated a conflict between two published sources. The booklet prints 286,470 as the Full Fee amount due at admission, where its own line items sum to 216,470, and its foreign column drifts by a few rupees in several places; the campus notice's totals all reproduce from its own line items. The notice is therefore preferred, and the adjudication rests on arithmetic rather than on judgement.

5.1.7. Reconstructed Programme Cut-offs

The serial-dictatorship simulation was run over the published 2083 Pulchowk priority applications. The 1,700 published rows reduce to 1,647 distinct applicants after quota deduplication, spanning merit ranks 1 to 7,179 and declaring on average 1.84 programme choices each. All 624 Pulchowk seats across sixteen programme–category pairs were filled, so every pair yields a cut-off. Table 21 reports the reconstruction.

Table 21. Reconstructed 2083 Pulchowk cut-offs: merit rank of the last candidate admitted

Programme	Regular		Full-fee	
	Seats	Last rank	Seats	Last rank
Computer	36	55	60	246
Electronics	24	110	24	303
Electrical	36	301	60	750
Mechanical	24	369	24	741
Aerospace	12	427	36	1,444
Civil	108	439	84	898
Chemical	12	900	36	2,933
Architecture	24	1,112	24	1,874
Total seats	**276**		**348**	

Figure 8 shows this analysis as a student receives it, for a stated merit rank of 240: the programmes that rank would and would not have cleared in 2083, each stated as last year's figure, followed by the ordering advice that strategy-proofness makes safe to give.

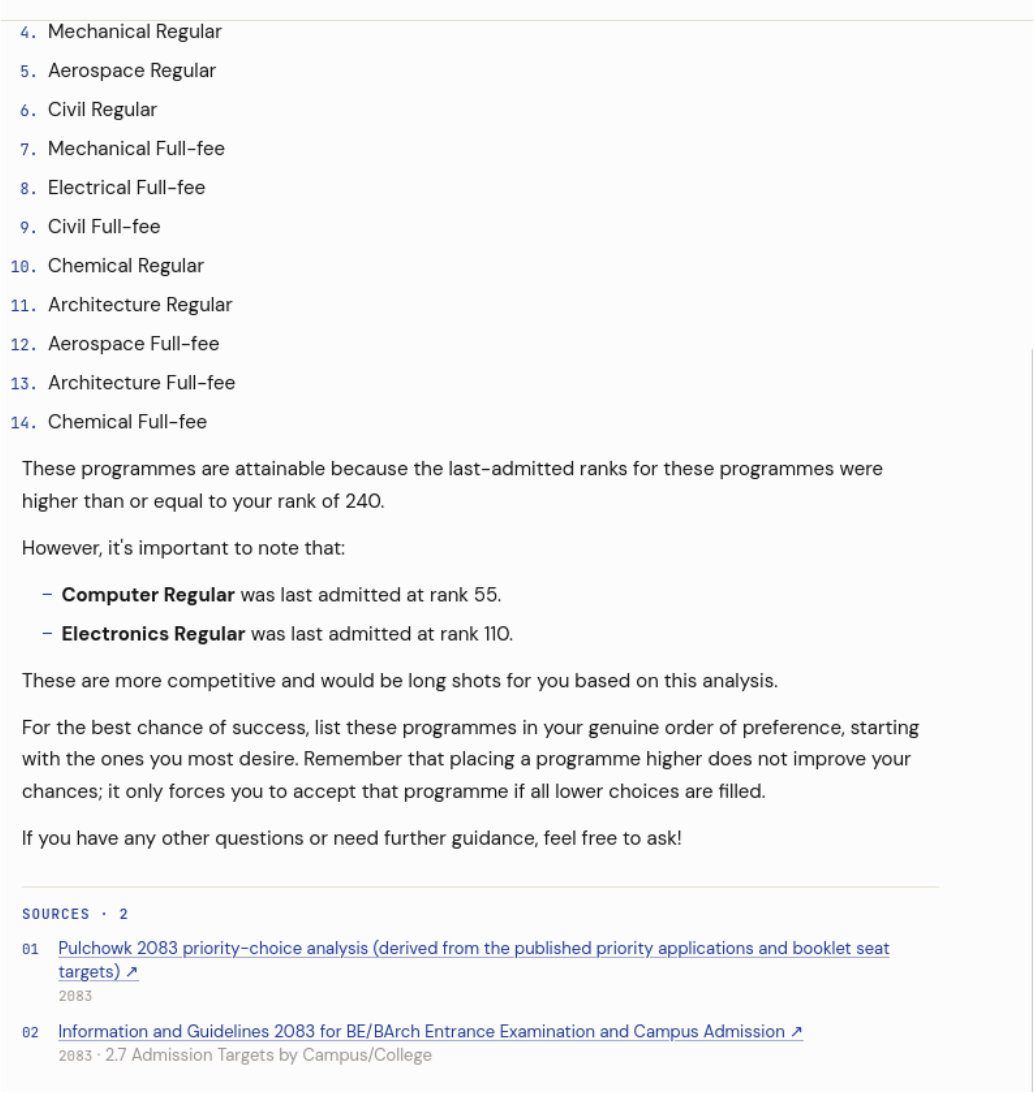

Figure 8. Priority guidance for merit rank 240, from the reconstructed 2083 cut-offs (transcript column only)

Two structural features of the result are worth noting. Computer Engineering is by a wide margin the most contested programme, its Regular cut-off at rank 55 being less than half that of the next programme; and for every programme without exception the Regular cut-off is sharper than the Full-fee cut-off, which is the empirical confirmation of the ordering advice the system gives – a Regular seat placed above its Full-fee counterpart costs a candidate nothing, because a candidate who does not reach Regular still falls through to Full-fee at the same rank.

5.1.8. Delivered Functionality

All eleven functional requirements of Table 7 are implemented in the delivered system. The backend comprises 3,695 lines across eleven modules and exposes fourteen endpoints; the frontend comprises three routes and fifteen components. The notice aggregator collects from six official sources and degrades per-source on failure. Conver-

sations persist across service restarts, and a reloaded transcript reproduces its citations, its refusals and its English-only prefaces exactly as they were streamed.

5.2. Discussion

5.2.1. What the Results Establish

The central claim of this work – that the reliability of an assistant in this domain is determined by the pipeline around the model rather than by the model – is supported by every measurement above. Not one improvement reported in Section 5.1 was obtained by changing the generator. The scope guard, the deterministic language handling, the computed fee totals, the reconstructed cut-offs, the grounding filter and the context-window correction are all pipeline changes, and the generator was identical before and after each.

A second and more specific finding is that the failures of a seven-billion-parameter model in this setting are systematic rather than random, and that being systematic they are engineerable. Each of the seven challenges in Section 4.7 had a mechanism, and in each case the mechanism was discoverable by measurement. The apparent loss of conversational memory had nothing to do with memory; the ineffective prompts were not ineffective but absent; the language failure was sustained by the very instruction intended to prevent it. This pattern – that the reported symptom is not the defect – recurred so consistently that the requirement to reproduce and measure before diagnosing became the most valuable element of the development methodology.

Third, the results support the specific architectural decision to remove consequential judgements from the model entirely rather than to instruct the model better. The comparison in Table 16 is the clearest instance: four increasingly elaborate prompt formulations produced 0/4, a refusal in the wrong language, 3/12 truncated replies, and a regression to 6/18, while deleting the instruction and removing its trigger from the input produced 24/24. The fee results make the same point arithmetically, and the priority reconstruction makes it structurally.

5.2.2. What the Results Do Not Establish

Several limits must be stated plainly.

The probe sets are small. Thirty-nine questions, twenty-seven follow-up cases, thirty-two live checks and eighteen graded answers are sufficient to establish that a specific failure mode was removed, and they are not sufficient to estimate an accuracy figure for the system as a whole. No claim of population-level accuracy is made anywhere in this report, and the figures in Section 5.1 should not be read as one.

The probe sets were also constructed by the developers, alongside the fixes they evaluate. This is standard for regression testing and it is not an unbiased evaluation design; a probe set built to catch a known failure will catch it. The strongest evidence in this chapter is consequently the negative results – the framings that scored 26/39 and 32/39, the four failed language formulations, the regression on bare follow-ups – since those were not the outcomes sought.

The grounding filter of Equation (16) is lexical and can therefore be deceived in both directions. An answer that paraphrases its source thoroughly may lose a citation it earned, and an answer that reuses distinctive vocabulary without depending on the passage may retain one it did not. The measured separation is wide enough for the filter to be useful, and the threshold is deliberately biased toward the less harmful error, but the filter is weaker than the trained and sampled methods reviewed in Section 2.2.3.

The cut-off reconstruction is exact with respect to its inputs and is not a prediction. It reproduces the 2083 allocation, and the system is required to present it as such, because cut-offs move each year with the applicant pool and the seat allocation. Its usefulness lies in informing the ordering of preferences, which strategy-proofness makes safe, and not in forecasting a threshold.

Finally, no user study was conducted. The system's usability, its comprehensibility to an anxious applicant, and its effect on the decisions such applicants make are all unmeasured, and all are the correct subject of future work.

5.2.3. Design Insights

Four insights generalise beyond this project.

Prompt position is a design parameter. With a model of this capacity, content nearest the question dominates. The computed fee block was ignored when placed ahead of the retrieved documents and effective when placed after them, and nothing else about it changed. The internal ordering of the block mattered similarly: totals had to precede components, because the model answers with the first plausible row it meets.

Naming a prohibited request keeps it alive. Every prompt formulation that mentioned the language request – including those that forbade it – kept the behaviour available. Removing the trigger from the input removed the behaviour.

A signal the pipeline has contaminated cannot audit the pipeline. Query rewriting exists to make a follow-up retrievable, which necessarily makes it domain-shaped. Measuring domain relevance on the rewritten query therefore measures the rewrite, not the question, and every off-topic message passed the test once rewritten.

Availability is not use. Citations assembled from retrieval describe what the model was

given, which for a refusal is precisely wrong. Attribution must be computed from the finished answer.

5.3. Comparative Analysis

No comparable public system exists for this domain, so comparison is made against the configurations this system replaced. Each row of Table 22 contrasts a baseline actually run against the final implementation, on the same probe set.

Table 22. Comparison of final implementation against the baselines it replaced

Capability	**Baseline**	**Baseline result**	**Final result**
Domain restriction	System-instruction rule only	Off-topic answered; the conversation corrupted thereafter	10/10 refused; follow-ups preserved
Scope classification	IN/OUT framing	26/39	36/39 (plus rescue)
English enforcement	Four prompt formulations	0/4 to 6/18	24/24
Fee arithmetic	Retrieved tables, model arithmetic	Wrong totals; an invented tuition table; an invented refund policy	20/20 totals verified
Prompt delivery	Default context window	Instruction silently discarded; 2,050 of 8,040 tokens evaluated	8,037 of 8,040 evaluated
Attribution	Cite everything retrieved	Sources printed beneath refusals	Filtered by measured use
Notice currency	Cache visible to the client only	Recent notices denied	Reported and linked from the digest
Social turns	Full retrieval pipeline	≈1.2 s of unusable retrieval per turn	Under 1 s, retrieval bypassed
Cut-off guidance	None available	Model estimated from seat counts	Exact reconstruction, 16 pairs

The pattern across the table is uniform. In every row the baseline is a configuration in

which the model was asked to do something – stay in scope, answer in English, compute a total, estimate a threshold, decide what it had used – and the final implementation is one in which that responsibility was moved into code, leaving the model to write the sentences. That reallocation, and not any change of model, is what the results measure.

Chapter 6. Conclusions and Recommendations

6.1. Conclusions

This project set out to build an assistant that answers questions about the IOE entrance examination and admission process from the official notices alone, that computes rather than generates every figure a student may act upon, that states which document each answer came from, and that declines plainly whatever the notices do not cover. That system was designed, implemented and evaluated, and it runs entirely on commodity hardware without contacting any external model service.

The following conclusions are drawn.

The corpus problem is tractable and its solution is durable. Thirteen official notices, translated once into structured English with their provenance recorded, index to 216 passages that answer the expository questions of this domain. Because the knowledge is external to the model, next year's admission cycle is accommodated by replacing documents and rebuilding the index, with no retraining and no code change – which is the property that makes retrieval augmentation the correct architecture here rather than merely a convenient one.

The consequential questions in this domain are not retrieval problems. Whether a form number appears on a pass list, what a Regular candidate pays over eight semesters, and how far down the merit list a programme admitted last year are questions with single correct answers. Answering them by nearest-neighbour search or by arithmetic performed inside a language model produces confident error. Answering them by exact lookup and deterministic computation, and handing the settled result to the model as authoritative context, produces correctness that can be verified: all twenty published fee totals reproduce from their line items, and the cut-off reconstruction resolves all sixteen Pulchowk programme–category pairs exactly.

Reliability was obtained from the pipeline, not from the model. No improvement reported in Chapter 5 came from changing the generator. Scope adjudication rose from 26/39 to 36/39 through a change of framing and to 26/27 through the addition of transcript context; English enforcement moved from 0/4 to 24/24 by deleting an instruction rather than by strengthening one; and the single most consequential defect found in the entire project was that the system instruction had been silently discarded by the inference server for want of an adequate context window. In each case the fix was structural.

Published administrative mechanisms can be reconstructed. IOE assigns seats by

serial dictatorship, and it publishes the merit order, the declared priority lists and the seat counts. The allocation is therefore deterministic and reproducible, and last year's cut-off for every Pulchowk programme – a table that exists nowhere in published form – was recovered exactly from 1,647 deduplicated applications. The result is strategy-proof to act upon, which is what allows the assistant to give ordering advice without giving a prediction.

The reported symptom is rarely the defect. A complaint of lost conversational memory was an out-of-scope answer contaminating every subsequent turn. A complaint of ineffective prompts was a prompt that was not present. A language failure was sustained by the instruction written to prevent it. The discipline of reproducing and measuring before diagnosing was the single most valuable element of the development methodology, and it is the element most likely to transfer to other work.

The limitations set out in Section 1.6 remain, and Section 5.2 states plainly what the measurements do and do not establish. The evaluation is regression evidence against specific failure modes, not a population-level accuracy estimate, and no user study was conducted.

6.2. Recommendations and Future Enhancements

6.2.1. Extending Coverage

1. **Ingest notice contents, not only listings.** Notices are published as scanned PDFs, so the assistant can presently report that a notice exists but not what it says. An optical character recognition stage over newly published notices, followed by translation and incremental indexing, would close the gap between the corpus and the live feed.
2. **Extend fee computation beyond Pulchowk.** The machinery is general; only Pulchowk publishes its schedule in full. As other campuses publish, their tables can be added with the same self-verification discipline.
3. **Extend the priority analysis to other campuses.** The simulation requires only published priority applications and seat counts. Pashchimanchal Campus already publishes an applicant priority form, and its inclusion would be a direct extension.
4. **Cover postgraduate admission.** The assistant acknowledges postgraduate programmes but the corpus does not support them.

6.2.2. Improving the Machinery

1. **Hybrid retrieval.** Dense retrieval alone underperforms on exact tokens – form numbers, notice identifiers, programme codes. Combining dense scores with a lexical

scorer such as BM25 under reciprocal rank fusion would improve precisely the queries a student is most likely to type verbatim.

2. **A cross-encoder reranker.** The present reranking is a single domain rule. A small cross-encoder scoring each query–passage pair jointly would order candidates far better than the bi-encoder that retrieved them, at a cost that may be affordable for six passages.

3. **A semantic grounding check.** The lexical filter of Equation (16) is deceivable by paraphrase. Sentence-level entailment between the answer and its cited passage would attribute more accurately, and would additionally support the sentence-level citation markers this system deliberately does not attempt.

4. **Answer caching.** Questions in this domain repeat heavily around deadlines. A semantic cache keyed on the embedded question, invalidated on re-index, would remove most generation cost at peak.

5. **Prompt-cache retention.** The system instruction is constant across turns and constitutes the largest single block of every prompt; retaining its evaluated state between turns would reduce time to first token materially.

6.2.3. Evaluation

1. **Construct a held-out benchmark.** A question set authored by applicants rather than by the developers, with reference answers drawn from the notices, would permit an accuracy estimate that the present probe sets cannot support.

2. **Adopt standard retrieval metrics.** Annotating relevant passages per question would allow recall@k, mean reciprocal rank and normalised discounted cumulative gain to be reported, making the retrieval configuration comparable with published systems.

3. **Conduct a user study across an admission cycle.** The measures that matter – whether applicants find answers they trust, and whether the assistant reduces missed deadlines – are behavioural and can only be obtained in deployment.

4. **Audit the translations.** Each translated document should be reviewed against its Nepali original by a reader independent of the translation, since a translation error propagates silently to every answer drawn from that passage.

6.2.4. Deployment and Operations

1. **Public hosting with observability.** A hosted deployment would require request logging, rate limiting and an operational dashboard, none of which a single-machine development setup needs.

2. **Scheduled ingestion.** Notice scraping and index refresh should run on a schedule rather than opportunistically on the first request after the cache expires.
3. **Nepali output, done properly.** The system answers only in English because the generator is not reliable in Nepali, and this is a genuine limitation for the population it serves. A Nepali-capable model, or a dedicated translation stage applied to the finished English answer with the figures held fixed, would remove it without reintroducing the failure mode of Section 4.7.2.
4. **Accessibility and mobile refinement.** Most applicants will reach this system on a handset. A formal accessibility audit and a mobile-first review of the three-column workspace are warranted before public release.

References

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, Mar. 2023.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS)*, Vancouver, Canada, Dec. 2020, pp. 9459–9474.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. 31st Int. Conf. Neural Information Processing Systems (NIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992.

[5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Online, Nov. 2020, pp. 6769–6781.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint* arXiv:2402.03216, Feb. 2024.

[7] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[8] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, Punta Cana, Dominican Republic, Nov. 2021, pp. 3784–3803.

[9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint* arXiv:2312.10997, Dec. 2023.

[10] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024.

[11] A. Abdulkadiroğlu and T. Sönmez, "School choice: A mechanism design approach," *The American Economic Review*, vol. 93, no. 3, pp. 729–747, Jun. 2003.

[12] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, Jan. 1962.

[13] S. Wollny, J. Schneider, D. Di Mitri, J. Weidlich, M. Rittberger, and H. Drachsler, "Are we there yet? A systematic literature review on chatbots in education," *Frontiers in Artificial Intelligence*, vol. 4, art. 654924, Jul. 2021.

[14] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, Dec. 2023, pp. 9004–9017.

[15] S. Timilsina, M. Gautam, and B. Bhattarai, "NepBERTa: Nepali language model trained in a large corpus," in *Proc. 2nd Conf. Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th Int. Joint Conf. Natural Language Processing (AACL-IJCNLP)*, Online, Nov. 2022, pp. 273–284.

[16] Qwen Team, Alibaba Group, "Qwen2.5 technical report," *arXiv preprint* arXiv:2412.15115, Dec. 2024.

[17] Institute of Engineering, Entrance Examination Board, Tribhuvan University, "Information and guidelines 2083 for B.E./B.Arch. entrance examination and campus admission," Lalitpur, Nepal, 2083 BS. [Online]. Available: https://entrance.ioe.edu.np/Notice/Detail/5128

[18] Institute of Engineering, Entrance Examination Board, Tribhuvan University, "B.E./B.Arch. entrance examination result 2083 – result publication notice and pass list," Lalitpur, Nepal, 2083 BS. [Online]. Available: https://entrance.ioe.edu.np/Notice/Detail/5131

[19] Institute of Engineering, Entrance Examination Board, Tribhuvan University, "Detailed syllabus of the B.E./B.Arch. entrance examination 2083," Lalitpur, Nepal, 2083 BS. [Online]. Available: https://entrance.ioe.edu.np/Notice/Detail/5127

[20] Pulchowk Campus, Institute of Engineering, Tribhuvan University, "B.E./B.Arch. admission 2083/084 – detailed notice: schedule, priority rules and fees," Lalitpur, Nepal, 2083 BS. [Online]. Available: https://pcampus.edu.np/2026/08/12/be-barch-admission-2083-detail-notice/

Appendix 1: Application Programming Interface

The backend exposes fourteen endpoints. Those under `/api/admin` require a bearer token supplied in the `X-Admin-Token` header; where no token is configured in the environment, every administrative route returns HTTP 503 rather than executing unauthenticated.

Table 23. Backend endpoint specification

Method	Path	Description
GET	`/api/health`	Liveness, active models and corpus size; no authentication, since it returns counts only
POST	`/api/chat`	Submits a question; returns a Server-Sent Events stream of `start`, `title`, `token`, `sources`, `error` and `done` events
POST	`/api/thread`	Mints a new conversation identifier
GET	`/api/threads`	Lists conversations for the calling browser, scoped by the `X-Client-Id` header
DELETE	`/api/threads/{id}`	Deletes one conversation
GET	`/api/history/{id}`	Replays a transcript with its citations
GET	`/api/today`	Current date in both calendars, for the masthead
GET	`/api/notices`	Aggregated notice feed, newest first
GET	`/api/deadlines`	Dated obligations, split into upcoming and passed
GET	`/api/admin/status`	Index and cache state
POST	`/api/admin/documents`	Uploads a Markdown document to the corpus, name- and size-constrained
DELETE	`/api/admin/documents/{name}`	Removes a document
POST	`/api/admin/reindex`	Rebuilds the vector index
POST	`/api/admin/notices/refresh`	Re-scrapes the notice boards

Appendix 2: Programme Codes and Seat Allocation

Table 24 gives the programme-code mapping used by the priority simulation of Section 4.4.8. Codes are those printed on the published priority-application form; seat counts are taken from section 2.7 of the published information booklet [17].

Table 24. Pulchowk programme codes, categories and seat allocation, 2083

Code	Programme	Category	Seats
1	Civil	Regular	108
2	Civil	Full-fee	84
3	Architecture	Regular	24
4	Architecture	Full-fee	24
5	Electrical	Regular	36
6	Electrical	Full-fee	60
7	Electronics	Regular	24
8	Electronics	Full-fee	24
9	Mechanical	Regular	24
10	Mechanical	Full-fee	24
11	Computer	Regular	36
12	Computer	Full-fee	60
27	Aerospace	Regular	12
28	Aerospace	Full-fee	36
29	Chemical	Regular	12
30	Chemical	Full-fee	36
	Total		**624**

Codes 13 to 26 are absent because they denote programmes offered at campuses other than Pulchowk, which the priority-application form enumerates on the same sheet but which fall outside the scope of this analysis.

Appendix 3: Representative Interaction Transcripts

The transcripts below illustrate the four principal paths through the orchestration graph. Assistant answers are reproduced as Markdown source; the interface renders them formatted.

A. Grounded retrieval with attribution.

Student	What documents do I need to submit for the quota?
Sarathi	Answer generated from retrieved passages of the quota-documents notice, naming the requirement set and the submission window, with each date given in both calendars.
Sources	Documents Required When Applying Under a Quota for BE/BArch Admission at IOE Constituent Campuses (2083)

B. Exact lookup over the pass list.

Student	Did form number 2083-4001 pass?
Internal	`[Pass list lookup: form number 2083-4001]` — exact record returned as labelled fields: status, form number, merit rank, name, district, alphabetical serial number. The scope guard passes the turn on the exact hit alone.
Sarathi	The record restated as an ordinary sentence, with the merit rank and district reproduced verbatim from the table.
Sources	BE/BArch Entrance Examination Result 2083 — published pass list

C. Priority guidance from the reconstructed cut-offs.

Student	My rank is 240. Which priorities should I fill for Pulchowk?
Internal	A priority block is produced; the pass-list lookup is instructed to skip the rank, since 240 is the student's own rank stated hypothetically and resolving it would return an unrelated candidate's identity.
Sarathi	Programmes cleared and not cleared at rank 240 in 2083, each stated as last year's figure, followed by the ordering advice: declare programmes in genuine order of preference, and place each Regular seat above its Full-fee counterpart.
Sources	Pulchowk 2083 priority-choice analysis (derived)

D. Refusal outside the domain.

Student	lets go on a holiday
Internal	No exact hit; the classifier returns NO; the relevance rescue scores the raw message below $\sigma = 0.60$. The `refuse` node is taken.
Sarathi	"I only handle questions about IOE admissions and the IOE entrance exam, so I have to leave that one alone. Ask me about the exam, your application, the fees, a result, or a campus and I can help."
Sources	None. Nothing was read, so nothing is cited.

The transcript is stored with the refusal in place, so that the conversation reloads identically and the following turn is not reasoned about as though a holiday had been discussed.